Lucrare de licență - Dinea Eduard-Emanuel.pdf

University of Bucharest

Document Details

Submission ID
trn:oid:::7314:396116313

Submission Date
Jun 28, 2026, 7:43 PM GMT+3

Download Date
Jun 28, 2026, 7:51 PM GMT+3

File Name
Lucrare de licență - Dinea Eduard-Emanuel.pdf

File Size
1.7 MB

57 Pages

18,828 Words

114,116 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Cited Text

Top Sources

0% Internet sources
0% Publications
1% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Hidden Text
265 suspect characters on 1 page
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Top Sources

0% Internet sources
0% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Submitted works
University Politehnica of Bucharest on 2020-06-25 <1%

2 Submitted works
University of Technology, Sydney on 2025-06-26 <1%

3 Submitted works
University of Bucharest on 2026-06-27 <1%

4 Submitted works
University of Bucharest on 2026-06-28 <1%

5 Submitted works
University of Bucharest on 2026-06-28 <1%

6 Publication
H. Conrad Cunningham. "Mapping component specifications to Enterprise JavaBe... <1%

7 Submitted works
Submitted on 1687922085721 <1%

8 Submitted works
Technical University of Cluj-Napoca on 2021-07-09 <1%

9 Submitted works
University of Technology, Sydney on 2025-06-26 <1%

10 Submitted works
Louisiana Tech University on 2017-06-26 <1%

11 Submitted works
West University Of Timisoara on 2025-07-22 <1%

4



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ARHITECTURA PAN CA MODEL PENTRU APLICAȚII WEB FULL-STACK

Studiu de caz privind reducerea risipei alimentare

Absolvent: Eduard Emanuel Dinea

3

Coordonator științific: Conf. univ. dr. Marius Iulian Mihăilescu

București, iulie 2026

3

Rezumat

Această lucrare propune și analizează arhitectura **PAN** (Prisma–Analog–Nx), un model full-stack pentru aplicații web moderne construit integral în ecosistemul TypeScript, în care straturile aplicației partajează tipuri și scheme de validare derivate dintr-o singură sursă de adevăr. Pentru a valida acest model, am proiectat și implementat *Kookio*, o aplicație web full-stack care servește drept studiu de caz al arhitecturii PAN. *Kookio* adresează un factor concret al risipei alimentare la nivel casnic — achiziția de alimente în absența unui plan al meselor — printr-un flux integrat de gestiune a cămării, planificare a meselor și derivare automată a listei de cumpărături, astfel încât achizițiile se raportează la mesele planificate, nu se acumulează redundant.

Motivația arhitecturală derivă din necesitatea unor stive web care să asigure coerență între frontend și backend, precum și un proces de dezvoltare unitar. PAN răspunde acestei nevoi nu prin reducerea numărului de unelte, ci prin integrarea lor sub un singur limbaj și prin formalizarea granițelor dintre componente.

Concret, fiecare strat al acestei stive unitare TypeScript acoperă o responsabilitate distinctă: randarea pe server (SSR) este asigurată de Angular prin meta-framework-ul Analog, comunicarea tipizată client–server prin tRPC, iar persistența datelor prin Prisma și CockroachDB. Organizarea proiectului într-un monorepo gestionat cu Nx impune o topologie aciclică a bibliotecilor, verificată la integrarea continuă (CI) prin regula `enforce-module-boundaries` a Nx, și permite eliminarea stratului de *repository* dintre procedurile tRPC și clientul Prisma.

Implementarea este validată prin teste unitare cu Vitest și prin testare cap-coadă (E2E) cu Playwright, însoțită de o evaluare empirică a latențelor stratului de persistență, a performanței front-end și a acoperirii testelor. Latența mediană măsurată (apeluri calde) a fost de 1,71 ms pentru CockroachDB și de 0,56 ms pentru Redis, iar optimizarea imaginilor reduce payload-ul livrat cu aproximativ 60 % pe întregul catalog. Evaluarea arată că stiva implementată atinge performanțe adecvate unui mediu de producție; coerența și extensibilitatea arhitecturii PAN sunt însă proprietăți structurale — susținute de topologia aciclică verificată la CI și de derivarea contractelor dintr-o sursă unică de adevăr —, dar nu consecințe ale acestor măsurători de latență.

1

Cuprins

1 Introducere **1**
1.1 Motivație . . . 1
1.2 Scopul lucrării . . . 2
1.3 Contribuții ale lucrării . . . 2

2 Preliminarii **3**
2.1 Motivația alegerilor tehnologice . . . 4
2.2 Arhitectura PAN . . . 6
2.3 Cadrul tehnologic . . . 9

3 Implementare **12**
3.1 Prezentare generală a implementării . . . 12
3.2 Implementarea aplicației . . . 14
3.3 Fluxuri funcționale și algoritmii produsului . . . 25
3.4 Metodologie și evaluare . . . 29

4 Concluzii **32**
4.1 Decizii arhitecturale . . . 34
4.2 Limitări și compromisuri ale arhitecturii . . . 36
4.3 Direcții viitoare . . . 37

Anexe **39**

Anexa A Demonstrarea aplicației *Kookio* **39**
A.1 Fluxul principal: de la configurare la gătit . . . 39
A.2 Detaliul pipeline-ului de conversie a unităților . . . 39

Anexa B Modelul de date **45**

Glosar de termeni **47**

Lista de acronime **50**

1 Introducere

1.1 Motivație

Risipa alimentară reprezintă o problemă cu impact economic și social documentat la nivel global și european. Conform raportului *UNEP Food Waste Index 2024* [27], gospodăriile au generat în 2022 peste 631 de milioane de tone de alimente irosite — echivalentul a peste un miliard de mese aruncate zilnic și reprezentând 60 % din risipa alimentară totală la nivel mondial. La nivelul Uniunii Europene, Eurostat estimează că fiecare locuitor a generat în medie 132 de kilograme de risipă alimentară în 2022, din care 54 % provine din gospodării [17]. Parlamentul European identifică între cauzele principale ale risipei la nivel de consumator: cumpărăturile impulsive, lipsa unei planificări adecvate a meselor și gestionarea deficitară a inventarului casnic de alimente [22]. Aceste cauze comportamentale sunt confirmate și de literatura academică: într-o sinteză sistematică a 60 de studii empirice asupra risipei alimentare la nivel casnic, Schanes et al. [31] identifică planificarea cumpărăturilor, verificarea inventarului înainte de cumpărături și comunicarea între membrii gospodăriei drept practici cu rol decisiv în prevenirea supra-cumpărării. Aceeași sinteză raportează că simpla utilizare a unei liste de cumpărături a redus risipa per capita cu aproximativ 20 %, și clasifică explicit aplicațiile care listează inventarul alimentar și planifică mesele printre măsurile de prevenție de tip tehnologic. Autorii subliniază însă că intenția de a reduce risipa nu se traduce automat în comportament (decalajul atitudine–comportament) și că o parte dintre cauze sunt structurale, dincolo de controlul individual [31]; prin urmare, o aplicație precum cea propusă în prezenta lucrare poate contribui la diminuarea achizițiilor redundante, fără a constitui o soluție completă a problemei.

În paralel, aplicațiile web moderne ridică provocări tehnice semnificative, documentate atât în literatura academică, cât și în sondajele anuale ale comunității de dezvoltatori. Sondajul *State of JavaScript 2024* identifică *complexitatea excesivă*, *supraîncărcarea cu opțiuni* și *modificările care rup compatibilitatea* drept principalele puncte de durere semnalate de utilizatorii framework-urilor frontend [6]. Aceeași sursă indică o adopție în creștere a randării pe server (SSR) — 59 % dintre respondenți o folosesc în 2024 — și o consolidare a TypeScript ca limbaj standard pentru aplicațiile de mare anvergură: conform sondajului *JetBrains Developer Ecosystem 2024*, adopția TypeScript a crescut de la 12 % în 2017 la 35 % în 2024, iar 67 % dintre dezvoltatori scriu predominant TypeScript [20]. Aceste dificultăți se accentuează în aplicațiile full-stack, unde integrarea deficitară a straturilor conduce la creșterea complexității și a costurilor de mentenanță pe termen lung.

Prezenta lucrare nu tratează aceste două direcții ca pe o relație de necesitate — problema risipei alimentare nu impune o anumită arhitectură —, ci o folosește pe prima drept domeniu-vehicul pentru a o investiga pe a doua: o aplicație casnică de gestiune a cămării, planificare a meselor și liste de cumpărături oferă un context full-stack reprezentativ (fluxuri integrate,

date derivate, persistență relațională) în care arhitectura PAN poate fi implementată și evaluată empiric.

1.2 Scopul lucrării

Mi-am propus să definesc și să evaluez arhitectura **PAN**, un model web full-stack bazat pe ecosistemul TypeScript, care integrează frontend-ul, backend-ul și stratul de persistență într-un proces unitar de dezvoltare. Pentru a o investiga empiric, am construit *Kookio*, o aplicație web care abordează problema risipei alimentare la nivel casnic. La nivel funcțional, aplicația acoperă gestiunea cămării, planificarea zilnică a meselor și derivarea automată a listei de cumpărături pentru utilizatori casnici. Astfel, *Kookio* servește drept studiu de caz pentru evaluarea arhitecturii PAN într-un context reprezentativ pentru aplicațiile web full-stack moderne.

În acest sens, lucrarea își propune:

- implementarea unui flux funcțional integrat pentru cămară, planificator zilnic și listă de cumpărături derivată, cu persistență într-o bază de date relațională;
- integrarea frontend-ului și backend-ului într-un proces de dezvoltare unitar, cu partajarea tipurilor și a mecanismelor de validare;
- analiza caracteristicilor și a proprietăților structurale ale arhitecturii PAN în ceea ce privește modularitatea (topologie aciclică între biblioteci) și mentenanța aplicației (absența unui strat de *repository*);
- construirea unui pipeline CI/CD reutilizabil, organizat pe workflow-uri și composite actions, aliniat la țintele Nx;
- validarea implementării prin teste unitare cu Vitest și prin testare cap-coadă (E2E) cu Playwright, însoțită de o evaluare empirică a latențelor stratului de persistență, a performanței front-end și a acoperirii testelor.

1.3 Contribuții ale lucrării

1. **Un model arhitectural PAN implementat:** Analog (Angular 21) cu randare pe server (SSR) peste Nitro, tRPC pentru API tipizat și Prisma către CockroachDB, organizat într-un monorepo Nx cu patru grupuri de biblioteci și topologie aciclică aplicată la CI prin regula `enforce-module-boundaries` a Nx.
2. **Aplicația *Kookio* implementată ca produs minim viabil (MVP):** aplicație web full-stack funcțională pentru utilizatori casnici, care acoperă integrat gestiunea cămării, planificarea zilnică a meselor, derivarea automată a listei de cumpărături și fluxul de *onboarding*

(configurare profil), cu autentificare prin Firebase, servind drept studiu de caz pentru arhitectura PAN.

3. **Tipare de partajare a contractelor de date dintr-o sursă unică de adevăr:** schemele Zod sunt generate automat din `schema.prisma` prin biblioteca `prisma-zod-generator` [5], iar tipurile TypeScript se derivă prin compoziție (`.pick()`, `.omit()`, `.extend()`, `z.infer<>`) din aceste scheme; contractele rezultate sunt reutilizate în interfața de utilizator, în rutele de server și în teste, fără un strat de *repository* între procedurile tRPC și clientul Prisma.

4. **Un flux automatizat CI/CD modular:** bazat pe workflow-uri reutilizabile și *composite actions* aliniate la țintele Nx, astfel încât verificările rulează doar pentru proiectele afectate de o modificare (`nx affected`).

5. **Metodologie de testare pe două niveluri distincte:** testare unitară cu Vitest și testare cap-coadă (E2E) cu Playwright, însoțită de practici concrete pentru pornirea la rece a serverului Vite (pre-încălzire în `global-setup`) și pentru navigarea către rute protejate (`waitUntil: 'commit'`).

6. **Evaluare empirică a stivei tehnologice implementate:** măsurători de latență la stratul de persistență (CockroachDB și Redis), de performanță front-end (Lighthouse și optimizarea imaginilor) și de acoperire a testelor, sintetizate în Tabelul 2.

Aplicația este implementată ca un MVP funcțional: recomandarea rețetelor pe baza inventarului din cămară este deja operațională prin filtrarea progresivă (Secțiunea 3.3.1), iar extinderile identificate vizează rafinarea acestei recomandări cu restricțiile dietetice colectate la *onboarding* și un mecanism de urmărire a consumabilelor de bază. Aceste extensii pot consolida contribuția aplicației la reducerea risipei alimentare prin diminuarea achizițiilor redundante. Detaliile privind implementarea sunt prezentate în Secțiunea 3, evaluarea empirică în Secțiunea 3.4, iar direcțiile concrete de evoluție în Secțiunea 4.3.

2 Preliminarii

Acest capitol prezintă contextul tehnologic al lucrării în trei subsecțiuni complementare. Secțiunea 2.1 expune motivația alegerilor tehnologice care au condus la formularea arhitecturii **PAN**, Secțiunea 2.2 definește arhitectura PAN și o compară cu alternative consacrate, iar Secțiunea 2.3 descrie tehnic fiecare componentă a stivei și rolul ei în aplicația *Kookio*.

2.1 Motivația alegerilor tehnologice

Pornind de la avantajele tipizării statice și de la consolidarea TypeScript ca limbaj standard pentru aplicațiile de mare anvergură — sondajul *JetBrains Developer Ecosystem 2024* indică o creștere a adopției de la 12 % în 2017 la 35 % în 2024, cu 67 % dintre dezvoltatori scriind predominant TypeScript [20] — în cadrul *Kookio* am explorat posibilitatea construirii unei aplicații complete, dintr-un capăt în celălalt, exclusiv pe TypeScript. Această stivă tehnologică va fi denumită convențional, în prezenta lucrare, arhitectura **PAN** (Prisma–Analog–Nx).

Avantajele tipizării statice nu sunt doar o preferință de inginerie, ci au fost cuantificate empiric. Studiul lui Gao et al. [18], care a evaluat sistemele *Flow* și TypeScript pe un eșantion de 400 de bug-uri publice extrase din proiecte reale de pe GitHub, arată că tipizarea statică ar fi putut detecta aproximativ 15 % dintre acestea (interval de încredere [11,5 %, 18,5 %] la nivelul de 95 %). Această valoare este însă o sub-aproximare deliberată: autorii măsoară exclusiv bug-uri publice, supraviețuitoare ale testării și *review*-ului, și exclud explicit beneficiile colaterale ale tipizării — documentarea, autocompletarea, navigarea în cod și siguranța la refactorizare [18]. Tocmai aceste beneficii nemăsurate constituie, în arhitectura PAN, o parte centrală a propunerii de valoare: o redenumire de câmp în `schema.prisma` se propagă, la compilare, în toți consumatorii (Secțiunea 3.2.4).

Același studiu cuantifică și costul tipizării — taxa de adnotare (*annotation tax*) — la 1,7, respectiv 2,4 token-uri per bug pentru *Flow* și TypeScript [18] (un *token* fiind o unitate lexicală a codului — identificator, cuvânt-cheie sau simbol). Acest cost redus este însă condiționat de ipoteza unei baze de cod deja adnotate, în care dezvoltatorul adaugă incremental doar adnotările aferente unei singure modificări. Arhitectura PAN deplasează acest compromis: contractele de tip la frontiera de date nu sunt adnotate manual, ci generate din `schema.prisma` prin `prisma-zod-generator` [5]. Schemele de bază nu se mai scriu de mână, iar singurul cost rămas la acest strat este derivarea subseturilor prin compoziție (`.pick()`, `.omit()`, `.extend()`) — o operațiune mecanică, cu un efort semnificativ mai mic decât adnotarea manuală a fiecărui contract. Astfel, dacă studiul demonstrează că beneficiul tipizării se menține chiar și sub un cost manual de adnotare, PAN elimină în mare măsură chiar acel cost.

O bifurcație arhitecturală anterioară alegerilor punctuale de stivă este aceea dintre o aplicație distribuită pe microservicii și un monolit modular livrat unitar. Analiza sistematică a literaturii industriale realizată de Soldani et al. [32] identifică, pe baza a 51 de studii, că durerile dominante ale arhitecturii pe microservicii sunt intrinseci distribuției: consumul de resurse de rețea (semnalat de 17 studii ca durerea operațională primară, întrucât microserviciile comunică prin apeluri API remote, nu *in-memory*), versionarea contractelor API între servicii, consistența datelor și tranzacțiile distribuite (21, respectiv 18 studii), precum și localizarea, coordonarea și monitorizarea instanțelor la rulare. Întrucât *Kookio* este o aplicație centrată pe date, destinată unei echipe mici și unei publicări unitare, am optat pentru un monolit modular în care aceste dureri nu apar prin

construcție: fluxul Analog → tRPC → Prisma execută apelurile către persistență în proces, fără un strat de *repository* și fără frontiere de rețea între componente (Secțiunea 3.2.2). Arhitectura PAN nu respinge microserviciile ca model inferior, ci ocupă deliberat un alt punct în spațiul de compromisuri. Este important de subliniat că studiul citat nu prezintă durerile microserviciilor ca defecte de proiectare, ci ca proprietăți intrinseci complexității distribuției, compensate de câștiguri reale la scară mare — Amazon, Netflix și Spotify fiind exemplele recurente. Comparația de față nu stabilește o ierarhie de valoare între cele două modele, ci delimitează spațiul de aplicabilitate: PAN este optim pentru aplicații centrate pe date, monolitice în sensul publicării unitare, dezvoltate de echipe mici — exact contextul în care durerile distribuției cântăresc mai mult decât câștigurile ei.

Partajarea contractelor de tip între capete nu este, în sine, o noutate: sistemele distribuite clasice o realizează de mult prin limbaje de descriere a interfețelor (*Interface Description Language*, IDL) precum *Protocol Buffers* sau *gRPC*, care generează cod tipizat pentru client și server dintr-o schemă comună. Particularitatea ecosistemului web este că această coerență a fost istoric greu de atins: clientul și serverul rulau adesea în limbaje diferite, iar frontiera dintre ele — serializarea JSON peste HTTP — este prin natura ei netipizată, astfel încât o modificare de contract pe server se manifesta pe client abia la rulare, sub formă de eroare. Ceea ce valorifică arhitectura PAN nu este, așadar, inventarea partajării de tipuri, ci faptul că un ecosistem unitar TypeScript — cu tRPC, care propagă semnăturile procedurilor fără un IDL separat, și cu scheme Zod generate din aceeași schemă de bază — aduce această garanție, deja familiară sistemelor distribuite, până la frontiera client–server a unei aplicații web și o verifică la compilare.

Alegerea bazei de date

O primă provocare conceptuală a fost alegerea bazei de date. Istoric vorbind, *MongoDB* a fost pentru mulți ani favoritul implicit al dezvoltatorilor din ecosistemul JavaScript/TypeScript, datorită stocării native în format JSON și a schemei flexibile care se mapează direct pe obiectele aplicației. Totuși, modelul de date al aplicației *Kookio* este intrinsec relațional: rețetele, ingredientele și utilizatorii sunt entități distincte legate prin relații de tip mai-mulți-la-mai-mulți, iar interogările compuse — legăturile de date, constrângerile de integritate referențială și tranzacțiile ACID — sunt esențiale pentru consistența datelor. Această caracteristică a modelului de date a făcut ca o bază relațională clasică să fie alegerea naturală pentru proiect.

Pentru abstractizarea limbajului bazei de date am ales Prisma [25] — o mapare obiect-relațională (ORM) care oferă siguranță a tipurilor de la capăt la capăt în TypeScript, generând clientul de acces direct din schema declarativă. Ca implementare concretă a serviciului m-am orientat spre CockroachDB [4], o bază de date distribuită compatibilă cu protocolul de legătură PostgreSQL [19] — astfel încât Prisma și întregul ecosistem de unelte funcționează nealterat. CockroachDB adaugă peste PostgreSQL scalare orizontală nativă, consistență puternică prin

algoritmul de consens Raft, precum și un nivel de servire fără infrastructură proprie gestionată (*serverless*), potrivit pentru un proiect academic care poate evolua către un mediu de producție.

Cadrul frontend și organizarea codului

Pentru interfața clientului, alegerea framework-ului Angular a venit firesc, dat fiind angajamentul față de TypeScript [10]. Pasul următor a fost identificarea unei opțiuni potrivite pentru codul ce urma să ruleze pe server. Aici am avut mai multe variante valide, majoritatea moștenitori conceptuali ai celebrei arhitecturi **MERN** (*MongoDB*, *Express*, *React*, *Node.js*), care în ultimii ani a migrat firesc către stive bazate pe Next.js și *Turborepo* (sistem de build pentru monorepo) — ambele dezvoltate de Vercel. Totuși, această direcție m-ar fi împins către ecosistemul *React* și sintaxa *TSX*, abandonând decizia inițială de a folosi Angular.

Pe parcursul cercetării am descoperit comunitatea **Analog** [7], un meta-framework care extinde Angular cu capabilități de server (randare pe server, rutare bazată pe fișiere, puncte finale API) — practic, echivalentul Angular al lui Next.js. Analog beneficiază de sprijinul direct al comunității **Nx** [12], o unealtă de gestionare a monorepo-urilor dezvoltată de compania Nrwl, fondată de foști ingineri ai echipei Angular de la Google. Această filiație explică de ce Nx are, în esența sa, ADN Angular și de ce oferă suport de primă mână pentru proiecte Analog: generatoare native, sistem de cache distribuit pentru sarcinile de construire și șabloane care accelerează crearea bibliotecilor specializate pe fiecare funcționalitate de bază a aplicației.

Astfel s-a conturat o paralelă structurală cu arhitectura **MEAN** (*MongoDB*, *Express*, Angular, *Node.js*) [21], arhitectura PAN reflectând maturizarea ecosistemului: Analog absoarbe rolurile pe care MEAN le distribuia între *Express* și Angular, iar Nx adresează o preocupare arhitecturală — organizarea monorepo cu cache distribuit — care, în perioada MEAN, nu era încă subiect de discuție. Această paralelă, împreună cu o comparație detaliată față de stiva *React* modernă (MERN + Next.js + Turborepo), este prezentată în Secțiunea 2.2.1.

De remarcat că PAN nu răspunde durerilor de complexitate semnalate în Secțiunea 1 prin reducerea numărului de unelte — stiva rămâne una bogată —, ci prin integrarea lor sub un singur limbaj și prin formalizarea granițelor dintre ele. Punctul de durere identificat de sondaje nu este numărul de unelte în sine, ci integrarea deficitară dintre straturi; PAN atacă exact această integrare, lăsând complexitatea structurată și verificată automat, nu eliminată.

2.2 Arhitectura PAN

PAN (Prisma–Analog–Nx) este o arhitectură full-stack monolitică modulară pentru aplicații web construite integral în ecosistemul TypeScript. Cele trei componente eponime — Prisma ca ORM declarativ, Analog ca meta-framework Angular cu suport SSR și Nx ca orchestrator de

monorepo — sunt elementele constitutive în jurul cărora se organizează o stivă coerentă cu cinci principii fundamentale:

1. **Limbaj unic de la capăt la capăt.** Întreaga aplicație — UI, server, validare, persistență — este scrisă în TypeScript. Această decizie elimină categoria de erori provenite din desincronizarea contractelor între limbaje diferite (de exemplu, Java pe server și TypeScript pe client) și permite reutilizarea uniformă a tipurilor.

2. **O singură sursă de adevăr pentru contractele de date.** Schema bazei de date declarată în `schema.prisma` este sursa primară din care se derivă, automat, atât clientul Prisma TypeScript, cât și schemele de validare Zod (via `prisma-zod-generator`). Tipurile aplicaționale sunt apoi compuse din aceste scheme prin operatorii `.pick()`, `.omit()`, `.extend()` și `z.infer<>`. O modificare în `schema.prisma` se propagă, la compilare, prin toate straturile.

3. **Topologie modulară aciclică, aplicată la CI.** Codul este organizat într-un monorepo Nx cu patru grupuri de biblioteci (`feature`, `prisma`, `server`, `shared`) și o relație de dependență strict aciclică. Regula `enforce-module-boundaries` a Nx respinge la integrarea continuă orice import care încalcă topologia, garantând că structura arhitecturală nu se degradează pe parcursul evoluției proiectului.

4. **Absența unui strat de *repository* intermediar.** Procedurile tRPC apelează direct clientul Prisma (`ctx.prisma`), fără un strat intermediar de tip *repository* orientat-obiect. Această decizie reduce numărul de straturi de adaptare și forțează ca logica de afaceri să se exprime în termenii primitivelor expuse de Prisma — un compromis explicit între simplitate și abstracție.

5. **Randare pe server ca mod implicit.** Combinația Angular + Analog livrează SSR drept comportament implicit pentru toate rutele. Astfel, prima impresie a utilizatorului depinde de un HTML populat încă de la prima cerere, important atât pentru indexarea SEO, cât și pentru perceperea performanței.

2.2.1 Comparație cu stiva React modernă

Stiva contemporană React (MERN + Next.js + *Turborepo*) ocupă același spațiu de soluții ca PAN, dar într-un alt ecosistem. Figura 1 rezumă vizual maturizarea de la MEAN la PAN, iar Tabelul 1 extinde comparația la stiva React modernă.

Diferențele majore între PAN și stiva React modernă sunt:

- **Comunitate.** PAN este construit pe Angular, framework cu o cotă de utilizare mai mică decât a React ($50{,}1\,\%$ față de $81{,}1\,\%$ *State of JavaScript 2024* [6]). Această diferență este

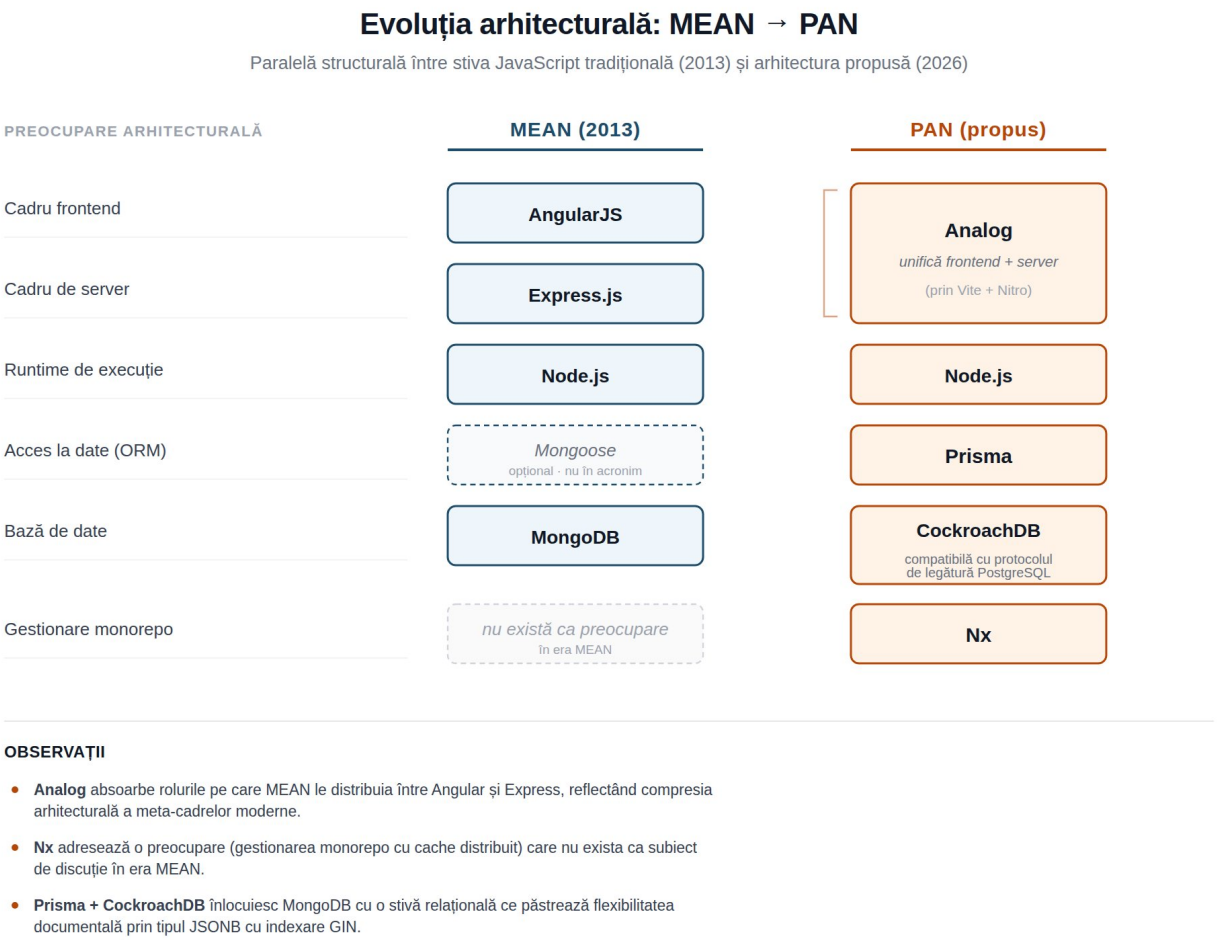


Figura 1: Paralelă structurală între arhitectura MEAN (Karpov, 2013) și arhitectura PAN propusă în prezenta lucrare. Diagrama evidențiază maturizarea ecosistemului: Analog absoarbe rolurile distribuite în MEAN între *Express* și Angular, iar Nx adresează preocuparea organizării monorepo care, în perioada MEAN, nu era încă subiect de discuție arhitecturală.

Tabelul 1: Comparație între MEAN (2013), stiva React modernă și arhitectura PAN.

Aspect	**MEAN (2013)**	**MERN + Next.js + Turborepo**	**PAN**
Frontend	Angular (vechi)	React	Angular 21
Meta-framework	— (Express)	Next.js	Analog
Server runtime	Node.js + Express	Node.js + Next.js	Nitro
API	REST manual	API routes + tRPC opțional	tRPC obligatoriu
Tipuri partajate	Nu	Parțial (App Router)	End-to-end automat
Bază de date	MongoDB	MongoDB sau PostgreSQL	CockroachDB
ORM	Mongoose	Prisma sau Drizzle	Prisma
Monorepo	—	Turborepo	Nx
Limbaj	JavaScript	TypeScript parțial	TypeScript pur

atât o limitare practică (mai puțini dezvoltatori disponibili, mai puține biblioteci de UI *third-party*), cât și o invitație la disciplină — Angular impune o organizare mai structurată implicit, prin injectarea dependențelor și componente *standalone* strict tipizate.

- **Maturitate.** Next.js are aproape un deceniu de evoluție și o adopție extinsă în producție; Analog are sub trei ani și o comunitate mică. PAN compensează prin alegerea Angular și Nitro (ambele mature individual) ca platforme de bază, dar acceptă riscul comunitar al Analog.
- **Tipizare end-to-end.** tRPC este în PAN o componentă obligatorie a arhitecturii, nu opțională. Această decizie face PAN mai restrictiv decât stiva React modernă (unde poți alege REST, GraphQL sau tRPC ad-hoc), dar elimină categoria de erori de desincronizare contractuală între client și server.

2.3 Cadrul tehnologic

2.3.1 Arhitecturi web moderne

Pentru o aplicație culinară în care prima impresie a utilizatorului depinde de un HTML populat încă de la prima vizită, modelul de randare pe client (CSR) al unei aplicații web de o singură pagină (SPA) Angular clasice nu era suficient. Cerințele de afișare rapidă, SEO pentru rețete publice și interactivitate ulterioară pe pagină au condus la o arhitectură care împarte sarcinile de randare între server și client [7, 14].

Strategiile de randare relevante pentru proiect sunt:

- **CSR** (randare pe client) — folosit în *Kookio* pentru ecranele autentificate (planificator, cămară), unde interactivitatea primează;
- **SSR** (randare pe server) — pentru pagini publice cu conținut dinamic (feed de rețete, profile), unde HTML-ul inițial trebuie să fie complet;
- **SSG** (generare statică de pagini) — prerandarea la *build* a paginilor cu conținut complet stabil; este suportată de Analog, dar nu este folosită în versiunea curentă a aplicației, în care și paginile statice sunt randate pe server;
- **arhitecturi hibride** — combinarea acestor strategii pe rute diferite, în funcție de natura conținutului [7].

Am ales o arhitectură full-stack hibridă în care frontend-ul și backend-ul rulează în aceeași bază de cod TypeScript. Decizia este motivată practic: tipurile și schemele de validare (tRPC cu validare prin scheme Zod) sunt partajate la nivel de modul, iar un singur pas de construcție livrează ambele straturi [28, 3].

2.3.2 Angular și randare server-side

Am ales Angular ca framework de frontend pentru că ecosistemul său oferă în mod nativ piesele de care aveam nevoie pentru *Kookio*: tipizare strictă, componente *standalone*, Angular Signals (semnale reactive) și setul de componente Angular Material/CDK, consistent în privința accesibilității [10, 9].

Suportul pentru SSR din *Kookio* provine din **Analog**, un meta-framework care extinde Angular cu:

- SSR (randare pe server) și SSG (generare statică de pagini) configurabile per rută,
- rutare bazată pe structura de fișiere,
- integrarea logicii de server (Nitro, tRPC) în același proiect [7, 8, 14].

Pentru o pagină de rețetă publică, SSR face diferența între un document HTML minimal până la hidratare (hydration) și un HTML populat încă de la prima cerere — important atât pentru utilizator, cât și pentru indexarea SEO pe care se sprijină distribuția organică a conținutului [7].

2.3.3 Monorepo și orchestrare cu Nx

Am organizat *Kookio* ca monorepo folosind **Nx**: o singură aplicație (`apps/web`) și un set de biblioteci segmentate pe responsabilități (`feature`, `prisma`, `server`, `shared`) [12].

Bibliotecile sunt grupate pe roluri:

- `feature/` — fluxuri de utilizator (`auth`, `shell`),
- `prisma-client` — schema, clientul generat, schemele Zod,
- `server/` — context tRPC, jurnalizare, middleware-uri, funcții utilitare de runtime server,
- `shared/` — UI, utilitare, date simulate (*mock*).

Peste această structură, Nx oferă două mecanisme esențiale folosite sistematic în proiect: `nx affected` rulează numai sarcinile (`lint`, `test`, `build`) atinse de modificările curente, iar regula `enforce-module-boundaries` respinge la CI importurile care încalcă topologia de biblioteci [12].

2.3.4 tRPC și partajarea tipurilor

Comunicarea client–server din *Kookio* folosește **tRPC** (*TypeScript Remote Procedure Call* — apeluri de proceduri la distanță tipizate pentru TypeScript): procedurile sunt declarate o singură

dată, în TypeScript, pe server, iar clientul Angular le invocă direct, fără un strat suplimentar de definiții API [28].

Avantajele concrete pe care le-am observat:

- inferența completă TypeScript elimină DTO-uri scrise manual care apar în arhitecturile REST clasice,
- validarea de intrare se face prin scheme Zod generate automat din schema Prisma [3, 5],
- refactorările prin redenumire de câmpuri propagă instantaneu pe ambele straturi.

Pentru *Kookio* aceasta s-a tradus, în practică, în eliminarea aproape completă a verificărilor de tip la frontiera client–server: contractul este compilatorul.

Limitele tipizării statice pure motivează, de altfel, stratul de validare la rulare. Taxonomia bug-urilor nedetectabile din [18] plasează erorile de conținut al șirurilor (*StringError* — URL-uri eronate, interogări malformate) pe locul al doilea ca frecvență, întrucât tipul `string` este opac pentru majoritatea sistemelor de tipuri statice. Validarea prin scheme Zod la frontiera tRPC acoperă tocmai o parte din acest gol, verificând la rulare formate, enum-uri și constrângeri pe care compilatorul TypeScript nu le poate exprima.

2.3.5 Persistență: Prisma și CockroachDB

Pentru persistență am ales **CockroachDB** ca bază de date și **Prisma** drept ORM [25, 4]. CockroachDB este compatibil cu protocolul de legătură PostgreSQL, ceea ce înseamnă că ecosistemul de unelte SQL (drivere, clienți `psql`, Prisma) funcționează fără adaptări, oferind în plus capacități native de scalare orizontală. PostgreSQL standard se concentrează pe replicare verticală, în timp ce CockroachDB adaugă peste protocol scalare orizontală nativă [4].

Prisma acoperă, în proiect, trei responsabilități: declararea schemei în `schema.prisma`, generarea unui client TypeScript cu tipuri exacte pentru fiecare model și gestionarea migrațiilor versionate aplicate la `deploy` [25].

La generarea schemelor Zod folosesc `prisma-zod-generator`, care expune o copie validabilă a fiecărui model — sursa de adevăr este schema Prisma, iar straturile API și UI consumă derivate ale ei [5].

2.3.6 Testare automată: Vitest și Playwright

Suita de testare din *Kookio* are două straturi distincte: teste unitare cu **Vitest** pentru logica izolată și testare cap-coadă (E2E) cu **Playwright** pentru scenariile complete printr-un browser real [16, 11].

Am ales Vitest pentru că rulează deja pe Vite (același bundler ca aplicația principală), ceea ce înseamnă că nu există configurări duplicate sau divergențe între `nx test` și `nx build` [16, 15].

Playwright acoperă fluxurile critice: autentificare, finalizare *onboarding*, adăugare în cămară, vizualizare planificator. Suita rulează în CI pe fiecare PR către `test` înainte de promovarea în `main` [11].

2.3.7 Managementul configurației și mediilor

Variabilele de mediu (chei Firebase, URL-uri Redis, secrete Vercel) sunt definite o singură dată în panoul Vercel, separat pe medii (`development`, `preview`, `production`) [33]. Această centralizare elimină fișierele `.env` adăugate accidental în istoricul Git și menține sincronizate toate ramurile.

Local, sincronizarea se face cu `vercel env pull`, care scrie un `.env.local` (exclus din versionare prin `.gitignore`) cu valorile mediului curent — niciun secret nu este versionat în Git [33].

3 Implementare

3.1 Prezentare generală a implementării

Capitolul precedent a stabilit alegerile tehnologice care formează arhitectura **PAN** (definită în Secțiunea 2.2). În cele ce urmează descriu cum se traduce acest tipar în structura efectivă a repository-ului: o singură aplicație Analog sub `apps/`, un set de biblioteci Nx grupate pe responsabilități sub `libs/` și un lanț unitar de execuție prin care o cerere ajunge de la runtime-ul server-side al Analog către straturile tRPC și Prisma.

3.1.1 Structura monorepo-ului Nx

```
kookio/
├── apps/
│   ├── web/                    # aplicație Analog (Angular 21 + SSR),
│   │                             rulează atât în browser cât și ca
│   │                             runtime server (Nitro)
│   └── web-e2e/                # suită Playwright pentru fluxurile
│                                 critice (login, onboarding, pantry,
│                                 planificator)
├── libs/
│   ├── feature/                # fluxuri vizibile utilizatorului
│   │   ├── auth/               #   login, înregistrare, sesiune
│   │   ├── onboarding/         #   pași de configurare profil
│   │   ├── pantry/             #   gestiune cămară
│   │   ├── plan/               #   planificator zilnic
│   │   ├── recipes/            #   feed și detaliu rețete
│   │   ├── groceries/          #   listă derivată de cumpărături
│   │   ├── saved/              #   colecții salvate
│   │   ├── dashboard/          #   ecran principal autentificat
│   │   └── shell/              #   layout, topbar, navigare
│   ├── prisma-client           # schemă Prisma, migrații,
│   │                             scheme Zod generate, tipuri derivate
│   ├── server/                 # auth, logger, trpc
│   │                             (helperi de runtime server)
│   └── shared/                 # ui, utility, mock
│                                 (componente, helperi, fixtures)
├── nx.json                     # configurație globală Nx
├── package.json                # manifestul principal al proiectului
├── tsconfig.base.json          # config TypeScript + alias-urile
├── vercel.json                 # rute, headers și build pentru Vercel
└── vitest.workspace.ts         # workspace Vitest (toate proiectele)
```

În practică, o modificare în `libs/feature/pantry` declanșează doar verificările pentru biblioteca afectată și pentru modulele care depind direct de ea (via `nx affected`), nu pentru întregul repository [12]. Topologia dintre grupuri (`feature/` nu importă din `feature/`, `shared/` nu depinde de nimic intern) este aplicată automat de regula `enforce-module-boundaries`, verificată la CI.

3.1.2 Fluxul aplicației cap-coadă

La nivel de ansamblu, o cerere parcurge trei straturi: *Analog (UI și SSR)* → *tRPC (API tipizat)* → *Prisma (acces la date)* [7, 28, 25]. Tranzițiile dintre straturi nu trec printr-un strat suplimentar de serializare manuală — tipurile TypeScript sunt partajate de la capăt la capăt, iar validarea de

intrare la frontiera tRPC folosește scheme Zod derivate din schema Prisma [28, 3, 5]. Detalierea pe etape a unei cereri concrete (de la SSR inițial până la apelurile ulterioare din browser) se găsește în Secțiunea 3.2.9, iar diagrama de secvență corespunzătoare în Figura 4. Imaginea de ansamblu a componentelor și a serviciilor gestionate extern este redată în Figura 2.

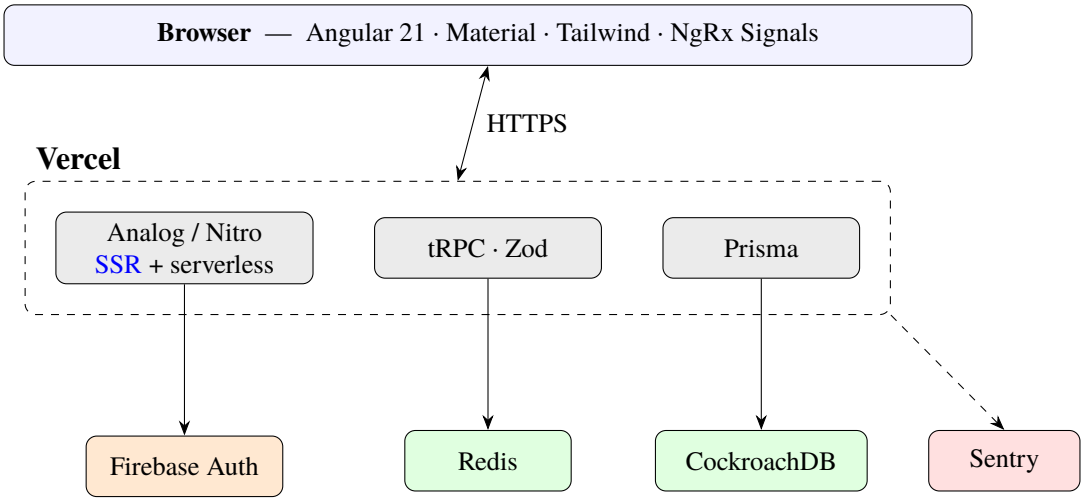


Figura 2: Arhitectura de ansamblu: browserul (Angular) comunică prin HTTPS cu platforma Vercel, care găzduiește runtime-ul Analog/Nitro (SSR + funcții serverless), stratul tRPC (cu validare Zod) și clientul Prisma. Persistența și serviciile transversale sunt gestionate extern: CockroachDB (date), Redis (cache, *rate-limiting*), Firebase (autentificare) și Sentry (monitorizarea erorilor).

3.2 Implementarea aplicației

Dacă secțiunea anterioară a justificat alegerile tehnologice, cea de față arată cum se vede fiecare alegere în repository: ce fișiere o materializează, ce convenții o aplică și ce constrângeri concrete a impus la nivelul codului. Pentru fiecare tehnologie, accentul nu mai cade pe *de ce* a fost aleasă, ci pe *ce a rezultat efectiv* odată integrată în *Kookio*.

3.2.1 Arhitecturi web moderne

Configurarea de randare a *Kookio* este concentrată în `apps/web/vite.config.ts`: plugin-ul `analog` este apelat cu `ssr: true`, fără rute prerandate. Astfel, SSR este modul de randare pentru toate rutele aplicației, inclusiv pentru paginile cu conținut stabil precum `/about`; SSG, deși suportată de Analog, nu este folosită, pentru ca fiecare răspuns să treacă uniform prin runtime-ul server [7].

Pe ramura SSR, distincția dintre rutele publice (`/recipes`, `/legal/attributions`) și cele autentificate (`/pantry`, `/plan`) nu se manifestă în strategia de randare, ci în ceea ce execută serverul înainte de a servi pagina: rutele autentificate trec printr-un *guard* (interceptor de rută) care

emite o redirecționare către `/login` dacă sesiunea Firebase lipsește, vizibilă pentru Playwright drept `ERR_ABORTED` pe Chromium și gestionată prin `waitUntil: 'commit'` în testele E2E.

3.2.2 Topologia bibliotecilor și absența unui strat de repository

Bibliotecile *feature* ale aplicației sunt felii verticale (*vertical slices*): fiecare reprezintă un flux utilizator complet (`auth`, `onboarding`, `pantry`, `plan`, `recipes`) și conține componentele, magazinele de stare (*store*-uri NgRx Signals) și serviciile de care depinde acel flux. Topologia impusă de regula `enforce-module-boundaries` a Nx este aciclică: `apps/` → `libs/feature/` → `libs/server/`, `libs/shared/`, `libs/prisma-client`, iar `libs/feature/X` nu poate importa din `libs/feature/Y`. Comunicarea între feature-uri se face prin magazine de stare `providedIn: 'root'` sau prin procedurile tRPC expuse de `libs/server/trpc` [12, 13, 28]. Graful de dependențe al proiectului, colapsat pe cele patru domenii de *scope*, este redat în Figura 3.

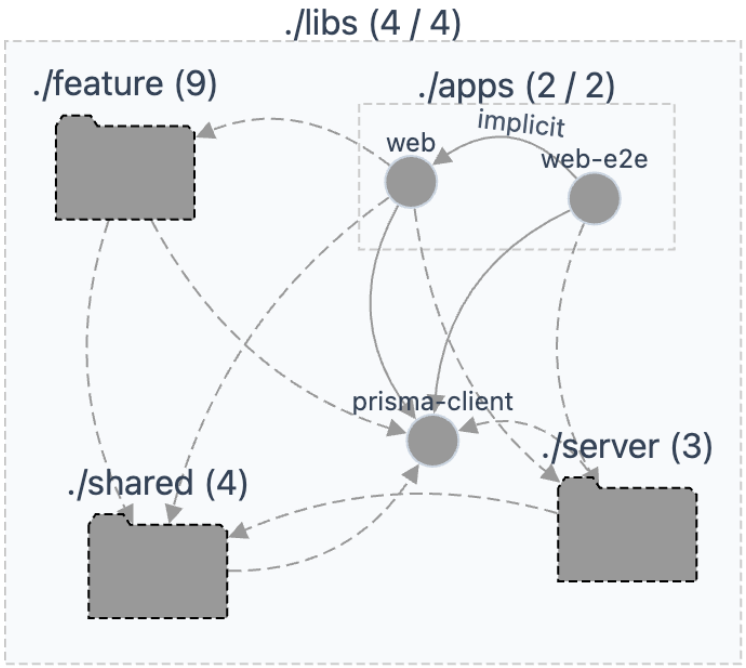


Figura 3: Graful de dependențe al proiectului (Nx), colapsat pe cele patru domenii de *scope*: `feature` (9 biblioteci), `server` (3), `shared` (4) și `prisma-client`, plus aplicațiile `web` și `web-e2e`. Săgețile redau relația de dependență; topologia este aciclică (`apps` → `feature` → `server`, `shared`, `prisma-client`), straturile inferioare neavând dependențe către cele superioare.

Organizarea bibliotecilor feature ca felii verticale (fiecare un flux complet de utilizator — `auth`, `pantry`, `plan`), și nu ca straturi orizontale corespunzătoare etapelor de procesare, reflectă concluzia centrală a lui Parnas: este aproape întotdeauna greșit să se înceapă descompunerea unui sistem pe baza unei diagrame de flux [23]. Parnas opune cele două modularizări ale sistemului

KWIC — una pe etape (input → shift circular → alfabetizare → output), cealaltă pe ascunderea deciziilor — și arată că în prima o schimbare precum formatul de stocare se propagă în fiecare modul, pe când în a doua rămâne confinată într-unul singur. Deoarece deciziile de proiectare transcend ordinea în timp a execuției, modulele nu corespund pașilor de procesare. În PAN, observația se traduce în interdicția structurală `feature/X` ↛ `feature/Y`: feliile comunică prin proceduri tRPC sau magazine de stare la nivel de rădăcină, nu pe orizontală prin etape.

Topologia aciclică impusă de regula `enforce-module-boundaries` a Nx corespunde, totodată, relației USES pe care Parnas o cere ca ordine parțială fără bucle [23]. Beneficiile pe care le atribuie acestei ordini se regăsesc în *Kookio*: straturile inferioare (`shared`, `server`, `prisma`) pot fi folosite fără cele superioare, iar „retezarea" feature-urilor superioare lasă un trunchi reutilizabil — exact proprietatea care permite adăugarea incrementală a unei noi biblioteci feature fără a atinge restul aplicației. Este însă important de marcat distincția pe care Parnas o face explicit: structura ierarhică (relația USES aciclică) și descompunerea „curată" (prin ascunderea informației) sunt două proprietăți dezirabile, dar independente. Regula `enforce-module-boundaries` a Nx o garantează pe prima — graful de dependențe rămâne aciclic —, dar nu garantează, prin sine, că fiecare bibliotecă ascunde efectiv o decizie de proiectare schimbabilă. Aceasta din urmă rămâne o disciplină aplicată la trasarea granițelor: regula CI previne degradarea topologiei, nu absența unui secret real într-o bibliotecă prost delimitată.

Codul backend și cel frontend coexistă în același repository: clientul Prisma și schemele Zod generate sunt localizate în `libs/prisma-client`, iar tipurile lor sunt importate direct în componentele Angular [5]. Nu există un strat clasic de tip *repository* orientat-obiect între procedurile tRPC și Prisma; dispecerii de rute apelează direct `ctx.prisma`, iar tranzacțiile multi-pas — inclusiv scrierile care ating mai multe domenii, precum transferul atomic al articolelor cumpărate din lista de cumpărături în cămară (`groceries.addBoughtToPantry`, Secțiunea 3.3.4) — folosesc `prisma.$transaction` [28, 26]. Această cuplare directă nu reduce testabilitatea procedurilor: contextul tRPC rămâne un punct de injecție prin care `ctx.prisma` poate fi substituit cu un dublu de test, mecanism detaliat în Secțiunea 3.2.10.

3.2.3 Angular și randare server-side

Rutele aplicației sunt definite pe baza structurii de fișiere (*filesystem-based*) sub `apps/web/src/app/pages/`: un fișier `about.page.ts` expune `/about`, `cook.[recipeId].page.ts` expune o rută dinamică cu parametru, iar un `cook.[recipeId].server.ts` colocat realizează încărcarea de date înainte ca Angular să randeze componenta [7, 8]. Fiecare `*.page.ts` este export implicit (*default export*) și este încărcat la cerere (*lazy loading*) de către dispecerul de rute Analog; nu există un fișier centralizat de rute.

Configurarea SSR la nivel de aplicație este minimală: `app.config.ts` înregistrează rutarea bazată pe fișiere, clientul HTTP și hidratarea cu reluarea evenimentelor capturate în intervalul

dintre SSR și hidratare (hydration) (mecanismul concret este descris în Secțiunea 3.2.9) [7]. Fiecare import adăugat în `app.config.ts` are însă un cost amplificat de SSR: modulul este evaluat pe traseul critic al fiecărei rute, iar fișierele agregatoare de tip *barrel* (re-exporturi de tipul `export *`) nu sunt eliminate de Vite în mediul de dezvoltare. Constrângerea practică este să se importe direct, prin sub-căi, doar tokenii efectiv utilizați, lăsând componentele grele (Material datepicker, autocomplete) să fie încărcate de la nivelul paginii care le folosește.

3.2.4 Gestionarea datelor cu Prisma și CockroachDB

Schema bazei de date se află la `libs/prisma-client/schema.prisma`, alături de migrațiile gestionate de Prisma și de generatorul `prisma-zod-generator`, care produce o copie validabilă a fiecărui model în formă Zod [25, 5]. Schema completă — cele 20 de modele și relațiile dintre ele — este redată ca diagramă entitate-relație în Anexa B. Tipurile derivate (seturi inițiale de date, intrări de formular, payload-uri pentru rute tRPC) sunt construite prin compoziție — `.pick()`, `.omit()`, `.extend()` — direct peste schemele generate, astfel încât o redenumire de câmp în `schema.prisma` să se propage automat în consumatori [5].

Două decizii structurale susțin acest aranjament. Întâi, indexul public `@kookio/prisma-client` este sigur pentru execuție în browser: exportă numai tipuri și scheme Zod, iar instanța `PrismaClient` este izolată sub sub-calea `@kookio/prisma-client/server`, iar importul acesteia este blocat de ESLint în codul de frontend [25]. Apoi, instanța Prisma este extinsă cu o reluare automată la conflict (*retry-on-conflict*) pentru codurile de eroare *serializable* specifice CockroachDB (`P2034`, `P2036`, `40001` și mesajul *restart transaction*); toate operațiile de scriere (`create`, `update`, `upsert`, `delete` și variantele `*Many`) trec prin acest *wrapper* (înveliș de cod), iar tranzacțiile multi-pas sunt declarate idempotente pentru a permite reluarea automată [26, 4].

Conexiunea concretă către CockroachDB este realizată prin tiparul *driver adapter* introdus în Prisma 7: instanța `PrismaClient` primește la construcție un adaptor explicit (`@prisma/adapter-pg`) care încapsulează *pool*-ul de conexiuni `pg` către protocolul de legătură PostgreSQL [25, 4]. Fondul de conexiuni este dimensionat pentru modelul de execuție *serverless*: cel mult cinci conexiuni per instanță, cu un *timeout* de conectare de 10 secunde și un *timeout* de interogare (`statement_timeout`) de 20 de secunde, astfel încât o instanță efemeră nu epuizează cota de conexiuni a bazei, iar o interogare blocată nu reține o conexiune la nesfârșit. Începând cu această versiune, Prisma renunță la motorul nativ scris în Rust: clientul rulează integral în JavaScript, iar URL-ul bazei de date este mutat din `schema.prisma` într-un fișier dedicat `prisma.config.ts`. Această separare între clientul tipizat (generat din schemă) și driverul efectiv de transport permite, în principiu, schimbarea sursei de date fără regenerarea clientului — o flexibilitate utilă pentru testare integrațională contra unei baze locale și pentru viitoare migrații între furnizori compatibili PostgreSQL.

Constrângerile de unicitate compusă (de exemplu `@@unique([userId, date, slotType])` pe `MealSlot`, sau perechea `userId–ingredientCatalogId` pe `PantryItem`) sunt declarate în `schema.prisma` și aplicate la nivelul bazei de date, iar Prisma generează automat formele cu cheie compusă (*compound key*, de tipul `where: { userId_ingredientCatalogId: { …} }`) pentru interogările de căutare aferente [25].

3.2.5 Construirea setului de date inițial

Datele cu care este populată baza — catalogul de ingrediente, categorii, arii culinare și cele 594 de rețete — nu sunt scrise manual, ci derivate dintr-o sursă publică, *TheMealDB*, printr-un pipeline offline de preluare și curățare. Sursa este însă o bază de date colaborativă (*crowd-sourced*): instrucțiunile sunt text liber, măsurile apar în forme eterogene (`"2 tbsp heaped"`, `"1/2 cup"`), iar denumirile ingredientelor conțin duplicate, forme de plural și greșeli de tastare. Curățarea acestor date la o formă conformă schemei Prisma cere transformări deterministe, versionate și reproductibile — motiv pentru care pipeline-ul este construit ca un set de scripturi TypeScript separate de aplicație, sub `research/the-meal-db/`, fiecare consumând artefactul fazei anterioare și producând un *snapshot* JSON.

Pipeline-ul se desfășoară în cinci faze. Fazele de *preluare* interoghează API-ul sursă — catalogul (categorii, arii, ingrediente) și rețetele, acestea din urmă descoperite prin scanarea celor 26 de litere ale alfabetului, cu până la 20 de poziții de ingredient parsate pentru fiecare rețetă din structura plată a sursei. Faza de *curățare a catalogului* normalizează denumirile la o cheie `snake_case` stabilă (diacritice eliminate, spații colapsate) folosită ca identificator unic în întreg pipeline-ul, și auto-clasifică fiecare ingredient într-una dintre categorii printr-o listă ordonată de douăsprezece reguli de potrivire (*first-match-wins*, cu `PRODUCE` drept regulă-fanion finală; ingredientele care nu se potrivesc rămân neclasificate, pentru revizuire manuală). Faza de *curățare și îmbogățire a rețetelor* derivă câmpurile structurate (descrise mai jos), iar o fază finală de *corecții reziduale* aplică, direct pe fișierele canonice, un set de rescrieri mecanice idempotente.

O decizie structurală a pipeline-ului merită evidențiată: separarea dintre ieșirea automată și sursa canonică. Scripturile de curățare nu suprascriu niciodată fișierul `recipes.json` comis în repository — ele scriu într-un snapshot intermediar (`clean-recipes.json`), iar promovarea în sursa canonică se face printr-o etapă deliberată de revizuire manuală. Astfel, rafinările făcute de mână (re-segmentări de pași pe care parserul automat le-a ratat, corecturi de transcriere) supraviețuiesc re-rulării pipeline-ului, în timp ce corecțiile pur mecanice rămân automatizate și idempotente. Sursa de adevăr este versionată, nu regenerată la fiecare rulare.

Trei dintre transformările de îmbogățire sunt euristici cu rol algoritmic, nu simple reformatări. Estimarea *numărului de porții* mapează fiecare ingredient la masa sa în grame — volumele prin densitatea apei, unitățile de tip bucată printr-o tabelă de greutăți per bucată — excluzând ingredientele de pură aromatizare (sare, ulei, oțet) și unitățile neglijabile; masa totală în vrac

se împarte la o constantă reglabilă de 400 g per porție — o greutate orientativă pentru o porție de fel principal, aleasă pragmatic și reglabilă, fără pretenția unei valori nutriționale — și se încadrează în intervalul $[1, 12]$, cu o valoare implicită de 2 porții pentru rețetele cu mai puțin de două ingrediente în vrac sau sub 200 g de date utilizabile. *Dificultatea* este un *bucket* în trei trepte (`EASY`, `MEDIUM`, `HARD`) derivat dintr-un scor de complexitate `ingrediente` $+ 2\times$ `pași` — pașii ponderați dublu, fiindcă reflectă efortul, nu doar lista de cumpărături — raportat la mediana scorurilor pe întregul catalog: sub `median` $\times\ 0{,}8335$ este `EASY`, peste `median` $\times\ 1{,}1665$ este `HARD`, restul `MEDIUM` (benzile sunt mediana $\pm\ 1/6$, deci dau o împărțire egală doar pentru o distribuție uniformă; complexitatea rețetelor fiind asimetrică la dreapta, rezultă mai multe `EASY` decât `HARD`). În fine, *parsarea măsurilor* transformă șirurile de text liber în triplete `(cantitate, MeasureUnit, note)` — intervalele iau marginea inferioară, iar unitățile nerecunoscute sunt mutate în câmpul de note, fără a pierde informație.

Faza de corecții reziduale este cazul în care realitatea datelor colaborative se manifestă cel mai clar: douăzeci și patru de transformări distincte, fiecare idempotentă (rulată a doua oară, nu mai modifică nimic). Mai degrabă decât o enumerare, ele se grupează în patru clase de probleme de calitate a datelor: *normalizare tipografică și Unicode* (apostrofuri tipografice și ghilimele curbe aduse la ASCII, fracții Unicode parsate, caractere de lățime zero eliminate); *integritate referențială* (rescrierea denumirilor de ingrediente de la plural la singular conform planului de îmbinare, astfel încât rețetele să se lege corect la catalog, plus deduplicarea ingredientelor repetate în aceeași rețetă pe care constrângerea `@@unique([recipeId, ingredientCatalogId])` le-ar respinge); *reconstrucția structurii de pași* (împărțirea pașilor care conțineau sub-pași numerați intern, eliminarea pașilor-antet și a metadatelor scurse ca instrucțiuni); și *regenerarea câmpurilor derivate* (slug-uri *kebab-case* din titlu, cu mapare explicită a caracterelor latine extinse, și reclasificarea dificultății după ce restructurarea pașilor a deplasat mediana globală). Caracterul idempotent este garantat structural — fiecare transformare potrivește doar forma necorectată sau se oprește când starea curentă coincide cu ținta — astfel încât pipeline-ul poate fi re-rulat în siguranță după orice rafinare manuală.

3.2.6 Comunicarea client–server cu tRPC

Dispecerii de rute tRPC sunt definiți sub `libs/server/trpc/src/lib/routers/`, câte un dispecer de rute per domeniu (`auth`, `pantry`, `onboarding`, `meal-plan`, `recipes`, `cooking-session` ș.a.). Fiecare procedură este compusă dintr-unul din trei *constructori*: `publicProcedure` (alias pentru `procedure`, expus anonim, fără autentificare, definit în `libs/server/trpc/src/lib/trpc.ts`), `protectedProcedure` (necesită sesiune Firebase verificată în `createContext`) și `onboardedProcedure` (necesită în plus `onboardingDone === true` pe utilizator, altfel respinge cererea cu `PRECONDITION_FAILED`), ultimii doi declarați în `libs/server/trpc/src/lib/middlewares/auth.ts` [28, 29]. Convenția este una de structură: o căutare cu `grep` prin constructorul folosit într-un dispecer de rute arată

imediat ce interfață expune punctul de acces, fără a fi nevoie să se inspecteze fiecare strat de *middleware* compus local.

Pe partea de client, instanța tRPC este creată în `apps/web/src/trpc-client.ts`: *ID token*-ul Firebase este atașat ca antet HTTP `Authorization` prin opțiunea `headers` a *link*-ului de transport (un *signal* reactiv citit la fiecare cerere), iar un `authAwareFetch` dedicat (i) împrospătează proactiv tokenul când expiră în mai puțin de cinci minute, (ii) reîncearcă cererea o singură dată cu un token nou la un răspuns `401`, (iii) rezolvă URL-urile relative la formă absolută în context SSR (Node.js nu are o bază implicită pentru `fetch('/api/…')`). Procedurile tRPC returnează `Observable`-uri RxJS în această configurație, iar consumatorii din Angular aleg între `from(…).pipe(…)` pentru flux reactiv și `await firstValueFrom(from(…))` pentru un singur rezultat [28].

3.2.7 Rute h3 auxiliare și infrastructura Redis

Pe lângă handler-ul tRPC (`apps/web/src/server/routes/api/trpc/[trpc].ts`), aplicația expune două rute `h3` suplimentare: `/api/health` pentru verificarea de stare (*healthcheck*) și `/api/internal/bench-db` pentru benchmark-ul de latență al CockroachDB (folosit în secțiunea de evaluare). Un singur *middleware* `h3` (`apps/web/src/server/middleware/security.ts`) acoperă două cerințe transversale: un `404` explicit pentru fișierele statice negăsite (`/assets/*`, `/fonts/*` și altele) — pentru a evita situația în care SSR-ul ar returna `200` cu HTML pentru un *bundle* înlocuit între deploy-uri — și respingerea cu `404` a tiparelor uzuale de scanare automată (`/wp-admin`, `.env`, `.git`, extensii `.php`). Antetele de securitate sunt aplicate pe două niveluri. La marginea rețelei (*edge*), prin `vercel.json`, se aplică HSTS cu durată de un an și `includeSubDomains`, care forțează accesul prin HTTPS pe întreg domeniul. Politica CSP este emisă per cerere de un plugin Nitro dedicat (`apps/web/src/server/plugins/csp.ts`): pentru fiecare răspuns HTML, plugin-ul generează un *nonce* unic, îl atașează scripturilor inline ale aplicației și îl include în antet, păstrând `script-src` strict (originea proprie plus nonce-ul) fără a permite execuția de cod inline arbitrar; restul surselor rămân limitate la originea proprie, cu o excepție de *frame* pentru domeniul de autentificare Firebase.

Componenta Redis se află în `apps/web/src/server/infra/` și este proiectată să tolereze indisponibilitatea. `getRedis()` este un singleton cu *circuit breaker* de 60 de secunde: după o conexiune eșuată, apelurile următoare eșuează rapid (*fail-fast*) în loc să recreeze clientul [30]. Pe ramura de citire (`groceries.list`), o funcție utilitară captează silențios eșecurile Redis și returnează mulțimea goală, astfel încât pagina să se randeze fără starea «cumpărat»; pe ramura de scriere, răspunsul include un câmp `persisted: false` dacă Redis este indisponibil, permițând clientului să păstreze comutarea optimistă (*optimistic update*) și să afișeze o notificare. Limitarea ratei (*rate-limiting*, `rateLimitFixedWindow`) folosește o fereastră fixă `INCR + EXPIRE` și

este aplicată atât pe ruta de *benchmark* (10 cereri pe fereastră), cât și pe handler-ul tRPC: 10 cereri pentru înregistrare (`auth.register`) și 300 pentru restul procedurilor. În acord cu principiul de toleranță la indisponibilitate descris mai sus, limitarea este *fail-open* — dacă Redis nu răspunde, cererea este lăsată să treacă în loc să fie respinsă, astfel încât o defecțiune a stratului de *cache* să nu blocheze autentificarea sau scrierile. Tot pentru a proteja instanța Redis de pe planul gratuit, o sarcină programată (*cron*) săptămânală apelează ruta `/api/cron/redis-keepalive`, menținând-o activă pentru a preveni ciclul de auto-suspendare.

3.2.8 Jurnalizare și observabilitate

Stratul de jurnalizare (logging) este izolat în `libs/server/logger/`: o singură instanță Pino este creată cu un transport dependent de mediu — `pino-pretty` cu *colorize* și marcaj de timp scurt în dezvoltare, JSON brut în producție (Vercel) [1]. Pe ramura de producție, instanța aplică o listă explicită de câmpuri redactate (*redact list*: `password`, `*.token`, `authorization`) pentru a împiedica scurgerea credențialelor în jurnale (*log*-uri).

Pe lângă instanța globală de jurnalizare, `createLogger(namespace, bindings)` returnează un *logger* copil cu o etichetă de *namespace* dintr-un set fix (`audit`, `http`, `trpc`, `prisma`) plus *binding*-uri suplimentare — de exemplu `createLogger('trpc', { router: 'pantry' })` produce `ns: 'trpc'` alături de `router: 'pantry'` pe toate intrările emise — iar `createAudit(bindings)` returnează un logger specializat pentru evenimente de audit (cu indicatorul `audit: true` aplicat automat). Mesajele sunt întotdeauna structurate — `logger.info({ userId, action }, 'message')` — nu interpolări de șir, pentru ca log-urile produse să fie filtrabile direct pe platforma de agregare.

Peste stratul de jurnalizare, monitorizarea erorilor în producție este asigurată de Sentry, integrat simetric pe client și pe server și inactiv (*no-op*) până la configurarea unei chei de ingestie (DSN), astfel încât mediile de dezvoltare și implementările anterioare configurării rămân neatinse. Pe client, un `ErrorHandler` global captează erorile netratate, iar un *helper* `reportError` izolat în `libs/shared/observability/` este apelat în punctele de tratare a erorilor din *store*-uri — blocurile `catchError` care consumă eroarea pentru a o transforma în stare de interfață —, făcând vizibile și eșecurile altfel tăcute. Pe server, funcția `wrap()` din `libs/server/logger/` devine punctul unic de captare: metodele `error` și `fatal` transmit câmpul `err` către `Sentry.captureException` înainte de a delega lui Pino, astfel încât fiecare apel `log.error({ err })` din routerele tRPC raportează automat, fără instrumentare la fiecare punct de eroare. Inițializarea clientului `@sentry/node` se face o singură dată, la pornirea serverului, printr-un *plugin* Nitro care înregistrează și un *hook* pentru erorile netratate, filtrat la coduri `5xx` pentru a nu raporta erorile de client `4xx`. Simetria este deliberată: erorile netratate sunt prinse de *handler*-ele globale pe ambele capete, iar cele tratate de cele două puncte unice de captare — `reportError` pe client, `wrap()` pe server. Pe lângă erori, clientul colectează și

tranzacții de performanță printr-o integrare de urmărire (*tracing*) cu o rată de eșantionare de 10 %, suficientă pentru a observa metricile vitale ale paginilor fără a depăși cota planului gratuit. La compilare, *plugin*-ul `@sentry/vite-plugin` încarcă hărțile sursă (*source maps*) în Sentry și le elimină din *bundle*-ul livrat: generate în mod *hidden*, fără comentariul `sourceMappingURL`, ele nu sunt servite niciodată clientului, iar fiecare implementare este etichetată cu o versiune (*release*) egală cu *hash*-ul *commit*-ului Git, astfel încât o eroare din producție indică exact *deploy*-ul care a introdus-o [2].

3.2.9 Fluxul de execuție al unei cereri

Detalierea fluxului rezumat în Secțiunea 3.1.2 se desfășoară pe trei etape.

Etapa 1 (randare inițială). Cererea HTTP ajunge la runtime-ul server-side al Analog (bazat pe Nitro), iar instanța Angular este executată pe server pentru a produce HTML-ul inițial [7, 14].

Etapa 2 (interogarea persistenței). Componentele Angular declanșează apeluri către procedurile tRPC ale stratului server-side; fiecare procedură își validează intrările cu o schemă Zod derivată din schema Prisma și folosește `ctx.prisma` pentru a interoga CockroachDB [28, 3, 5, 24]. Rezultatele se propagă înapoi prin stratul tRPC către componentele care participă la randarea inițială.

Etapa 3 (hidratare și interacțiune ulterioară). HTML-ul randat ajunge la browser, iar instanța Angular preia controlul prin hidratare (hydration) — arborele DOM produs pe server este reutilizat, iar evenimentele capturate în interval sunt rerulate de `withEventReplay()` [7]. De aici, interacțiunile ulterioare apelează direct procedurile tRPC din browser, fără o nouă randare pe server [28]. Figura 4 redă acest flux ca diagramă de secvență.

3.2.10 Testare și validare

Suita de Vitest este orchestrată de `vitest.workspace.ts` la rădăcina repository-ului, care colectează automat toate fișierele `vite.config.*` și `vitest.config.*` din proiectele Nx [16, 15]. Fișierele de specificații (*spec*-uri, `*.spec.ts`) sunt colocate lângă codul testat, atât în bibliotecile de *feature*, cât și în bibliotecile de server (de exemplu `pantry.router.spec.ts` alături de `pantry.router.ts`). Granularitatea este principiul director: `nx affected -t test` rulează numai spec-urile atinse de modificările curente.

La nivelul bibliotecilor de server, procedurile tRPC se testează izolat, fără a porni un server HTTP și fără o bază de date reală: fiecare spec construiește un context fals în care `ctx.prisma` este un dublu de test cu metode `vi.fn()` declarate exact pentru operațiile atinse de procedură, iar procedura se invocă prin `appRouter.createCaller(ctx)` [28, 16]. Întrucât clientul Prisma ajunge la proceduri prin context, nu printr-un import direct, contextul preia rolul de izolare pe

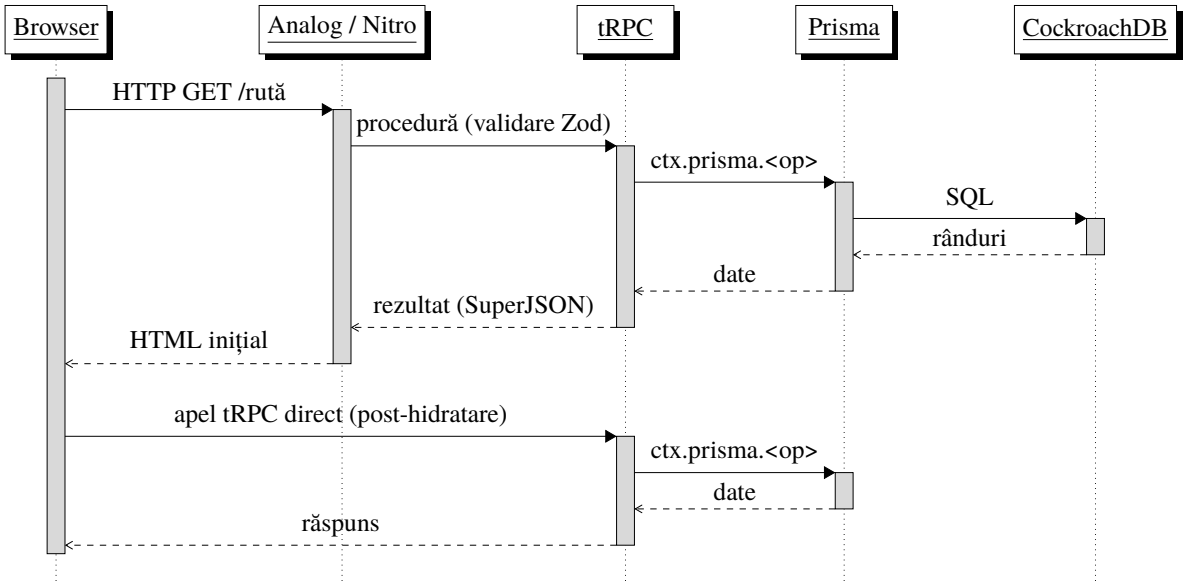


Figura 4: Diagrama de secvență a unei cereri. **Randare inițială (SSR):** cererea HTTP ajunge la runtime-ul Analog/Nitro, care interoghează persistența prin straturile tRPC (cu validare Zod) → Prisma → CockroachDB și returnează HTML-ul inițial. **Interacțiune ulterioară:** după hidratare, apelurile pleacă direct din browser către tRPC, fără o nouă randare pe server.

care l-ar fi avut un strat de *repository*: absența acestuia (Secțiunea 3.2.2) nu sacrifică testabilitatea, ci doar mută punctul de injecție în context.

Suita Playwright se află la `apps/web-e2e/` și are propriul `global-setup.ts` care preîncălzește toate rutele aplicației înainte ca primul proces de lucru (*worker*) să pornească — măsură necesară pentru a evita constrângerile de timp la prima pornire pe Vite în mediul de dezvoltare [11]. Fluxurile critice (autentificare, înregistrare, finalizare *onboarding*, adăugare în cămară, vizualizare planificator) parcurg interfața în Chromium; rutele protejate folosesc `waitUntil: 'commit'` pentru a evita eroarea `ERR_ABORTED` produsă de redirecționările Angular în timpul hidratare (hydration).

Disciplina cea mai vizibilă în practică se aplică la refactorizările care modifică dependențele injectate într-o componentă partajată: testele bibliotecii respective continuă să treacă (stub-urile locale — *stubs*, înlocuitori de test — sunt actualizate), dar spec-urile modulelor dependente pot eșua cu `NullInjectorError`. Pentru a detecta aceste regresii, se rulează `nx affected -t test`, care urmărește graful de dependențe Nx și rulează tot ce este afectat [12].

3.2.11 Dezvoltare și deploy

Configurarea mediilor este centralizată în panoul Vercel, separat pe `production`, `preview` și `development` [33]. Local, `vercel env pull` sincronizează valorile mediului curent într-un `.env.local` (exclus din versionare prin `.gitignore`), eliminând necesitatea ca vreun secret să fie versionat în Git.

Promovarea între medii respectă fluxul `dev` → `test` → `main`. Ramura `dev` alimentează mediul de previzualizare (*Preview*) de dezvoltare; ramura `test` declanșează suita E2E pe mediul de previzualizare de testare înaintea promovării în `main`, iar ramura `main` alimentează publicarea (*deploy*) de producție în regiunea `fra1` (Frankfurt) [33].

Build-ul de producție rulează `nx build web` cu `NODE_OPTIONS=--max-old-space-size=6144`. Limita implicită a heap-ului V8 este insuficientă pentru *bundling*-ul integral al stivei Angular Material în context Nitro: fazele de transformare a arborelui sintactic abstract (AST) și de generare a hărților sursă (*source maps*) acumulează volume mari de obiecte temporare, iar build-ul eșuează cu `JavaScript heap out of memory` înainte de finalizare. Ridicarea limitei la 6 GB asigură marja necesară pentru ambele faze. Prisma Client este regenerat în fluxul automatizat (*pipeline*) — ținta `generate` este dependență a țintei `build` —, astfel încât clientul TypeScript și schemele Zod derivate din `schema.prisma` sunt întotdeauna sincronizate cu modelul de date înainte de compilarea consumatorilor [25]. Întrucât Prisma 7 nu mai distribuie un motor nativ dependent de platformă (clientul rulează integral în JavaScript prin *driver adapter*), regenerarea privește exclusiv codul tipizat, nu binare specifice mediului de execuție.

3.2.12 Pattern-uri arhitecturale descoperite în implementare

Pe lângă principiile structurale stabilite *a priori* (Secțiunea 2.2), implementarea aplicației *Kookio* a expus trei tipare arhitecturale care nu erau predictibile din alegerile inițiale, dar care s-au consolidat ca lecții generalizabile pentru orice proiect PAN. Spre deosebire de limitările intrinseci ale stivei (Secțiunea 4.2), pattern-urile descrise aici sunt *soluții* reutilizabile, codificate în repository-ul de ghiduri al proiectului pentru a evita reapariția aceleași probleme:

- **Încărcătoarele SSR autentificate cer hidratare client-side a datelor autentificate.** La prima descoperire, încărcătoarele `*.server.ts` colocate cu rutele dinamice apelau direct proceduri tRPC autentificate prin `appRouter.createCaller(ctx)`, presupunând că sesiunea Firebase din `createContext` este disponibilă atât la primul SSR, cât și la navigarea ulterioară pe partea de client. Ipoteza era falsă: pe navigare pe partea de client, Analog execută *loader*-ul ca un `fetch` obișnuit care nu propagă antetul `Authorization` cu *ID token*-ul Firebase, iar procedurile autentificate răspund cu `401`. Pattern-ul corect, codificat în repository: *loader*-ul încarcă doar date publice (*recipes.byId*, *about page content*), iar datele autentificate (disponibilitatea ingredientelor în cămară, starea de „salvat” pentru o rețetă, preferințele utilizatorului) sunt hidratate ulterior, din browser, prin clientul tRPC cu `authAwareFetch`. Pe planul aplicabilității generale, pattern-ul se reduce la observația că SSR și autentificarea cu *Bearer token* nu coabitează gratuit în nici un meta-framework hibrid — orice cititor care evaluează PAN pentru un proiect propriu trebuie să auditeze această dimensiune înainte de a se baza pe *loader*-i SSR pentru date sensibile.

- **Numele rutelor în routere bazate pe fișiere trebuie să fie semantice, nu numerice.** La construirea paginii pentru codul de eroare `500`, alegerea naturală a fost o rută `/500` (paralelă cu `/404` deja funcțională). La rulare, ruta nu se activa: serverul răspundea cu `200` și HTML-ul paginii principale. Cauza, descoperită prin investigare graduală, a fost o particularitate a router-ului bazat pe fișiere al Analog: segmentele *all-numeric* sunt interpretate ca parametri dinamici nevalizi, iar *matcher*-ul cade silentios prin lista de rute fără a semnala o eroare de configurare. Pattern-ul corect, codificat în repository: rutele numite *semantic* (`/server-error`, `/not-found`, `/maintenance`) în locul codurilor numerice. Pe planul aplicabilității generale, pattern-ul se reduce la observația că orice convenție de routing bazată pe fișiere trebuie să separe *coduri numerice* (artefacte de protocol HTTP) de *nume de rute* (identificatori semantici pentru utilizator) — chiar dacă, în lumea protocoalelor, cele două par interșanjabile.

- **O singură euristică defensabilă pentru estimarea cantităților, doi consumatori.** Pipeline-ul offline de curățare a datelor (etapa de *seed*) și logica de runtime (decrementarea cămării, derivarea listei de cumpărături) consumau inițial euristici de conversie aproape identice, scrise separat — o tabelă manuală `INGREDIENT_PIECE_GRAMS` la *seed* și *carve-out*-uri proprii la runtime —, ceea ce ramifica fiecare extensie și producea, în timp, divergențe de comportament. Consolidarea celor două într-o singură sursă de adevăr (perechea de funcții pure `toGrams`/`fromGrams` și coloana `IngredientCatalog.pieceGrams` din schema bazei de date) este detaliată în Secțiunea 3.3.3. Pe planul aplicabilității generale, pattern-ul se reduce la o regulă de proiectare: *o singură euristică defensabilă, doi sau mai mulți consumatori, niciodată o euristică per consumator*. O critică a estimării (de exemplu, „densitatea apei este o aproximare slabă pentru uleiuri”) se aplică simultan tuturor consumatorilor; o îmbunătățire se propagă în același mod.

Cele trei pattern-uri împărtășesc o structură comună: fiecare a apărut din *frecuș de implementare* (*loader*-ul SSR care răspundea cu `401`, ruta `/500` care nu se activa, două euristici aproape identice scrise separat), iar soluția nu a fost o adăugare de funcționalitate, ci o *re-așezare* a piesei deja existente la un nivel mai apropiat de invariantul real al problemei. Pe planul auditului arhitectural, acest tipar de descoperire — *frecuș* → *lecție* → *codificare* — a fost observat în literatura de inginerie software ca o caracteristică a sistemelor evolutive, în care invariantul corect este învățat din experiența implementării, nu impus din proiectul inițial.

3.3 Fluxuri funcționale și algoritmii produsului

Secțiunile anterioare au descris cum stratul arhitectural — Analog, tRPC, Prisma — materializează arhitectura PAN la nivel de cod. Această secțiune coboară un nivel mai jos, către *algoritmii*

produsului care realizează propunerea de valoare a aplicației: filtrarea progresivă a rețetelor pe baza inventarului din cămară, decrementarea tranzacțională a cantităților după gătit și derivarea automată a listei de cumpărături din slot-urile planificate. Cele trei fluxuri converg într-un singur *algoritm de estimare* care unifică pipeline-ul de curățare a datelor (descris în Secțiunea 3.2.4) cu logica de execuție din producție. Filtrarea progresivă a rețetelor pe baza inventarului din cămară (Secțiunea 3.3.1) și derivarea automată a listei de cumpărături (Secțiunea 3.3.4) operaționalizează două dintre strategiile de prevenție identificate de Schanes et al. [31]: gătitul pe baza a ceea ce este deja stocat acasă și verificarea inventarului înainte de cumpărături.

3.3.1 Filtrarea progresivă a rețetelor

Pe ruta `/recipes/match`, utilizatorul vede catalogul de rețete partiționat în trei niveluri, etichetate în interfață *Alright* (toate ingredientele non-neglijabile sunt prezente în cămară), *Almost* (lipsesc unu până la trei ingrediente non-neglijabile) și *All* (toate rețetele publice, fără filtrare). Procedura tRPC `recipes.match` primește un câmp `level` cu exact aceste trei valori (`'Alright'`, `'Almost'`, `'All'`), alături de un `cursor` opțional și un `limit` (implicit 20, maximum 50), și returnează o pagină de rețete cu metadatele de potrivire necesare interfeței: numărul de ingrediente lipsă (`missingCount`), numele primelor trei dintre ele (`missingNames`) și numărul total de ingrediente al rețetei (`totalIngredients`). Pagina poartă și un `nextCursor` pentru paginare și un indicator `partialScan` folosit de client pentru a continua automat o scanare întreruptă de bugetul de examinare.

Algoritmul este, în esență, o *diferență de mulțimi* între ingredientele rețetei și conținutul cămării utilizatorului. Pentru fiecare rețetă din catalogul `PUBLIC`, procedura iterează prin lista de `RecipeIngredient` și sare peste ingredientele cu unitatea neglijabilă (`TO_TASTE`, `TO_SERVE`, `GARNISH`, `PINCH`) — sare, piper, garnitură de pătrunjel și ulei nu pot bloca o rețetă din nivelul *Alright* doar pentru că lipsesc din evidența explicită a cămării. Restul ingredientelor sunt căutate într-o hartă construită din `pantryItem.findMany({ where: { userId } })`, comparându-se la nivel de *existență* (prezența ingredientului în cămară, indiferent de cantitate). În varianta livrată în lucrare, comparația de cantități nu este efectuată; un fals pozitiv rar este preferat unei logici de unitate care duplică pipeline-ul descris în Secțiunea 3.3.3.

Decizia de a păstra logica de potrivire simplă — *prezență*, nu *cantitate suficientă* — a fost luată deliberat: costul unui fals pozitiv este redus (o rețetă apare ca gătibilă deși utilizatorul are o cantitate insuficientă dintr-un ingredient), iar varianta cantitativă ar duplica logica de conversie a unităților din pipeline-ul de grame (Secțiunea 3.3.3) la frontiera de potrivire, fără un câștig proporțional în acuratețea percepută. Pentru o evoluție viitoare, varianta cantitativă se construiește direct peste acel pipeline — ingredientele se normalizează la grame înainte de comparație, iar nivelul *Alright* cere ca pantry-ul să acopere consumul în grame al rețetei pentru toate ingredientele non-neglijabile.

3.3.2 Sesiunea de gătit și decrementarea tranzacțională

Pe ruta `/cook/:recipeId`, utilizatorul parcurge pașii rețetei și, la final, un dialog îi propune o listă de cantități pre-completate (proporțional cu numărul de porții selectat) pentru confirmare. La trimitere, clientul apelează `cookingSession.complete`, care decrementează cămara într-o singură tranzacție Prisma (`$transaction`). Pentru fiecare `(ingredientCatalogId, consumedQuantity)` confirmat, procedura caută rândul corespunzător prin cheia compusă `(userId, ingredientCatalogId)` (constrângere `@@unique` declarată în Secțiunea 3.2.4), scade cantitatea consumată din cantitatea existentă și — aici intervine algoritmul de estimare descris în secțiunea următoare — șterge rândul dacă cantitatea rămasă coboară sub un prag minim (`0,5` g, pragul de zgomot al unei ciupituri de sare). Tot demersul se desfășoară atomic: ori toate decrementările trec și `cookingSession.completedAt` este actualizat, ori nimic nu se aplică.

Două detalii de implementare merită menționate. Primul, decrementarea se face în grame intern (nu în unitatea de afișare a rândului din cămară), pentru a permite consumul în unități diferite de stocare — de exemplu, o rețetă cere `200 g` de făină, dar utilizatorul a introdus făina ca `0,5 kg`. Conversia înapoi la unitatea de afișare se face la final, înainte de `prisma.update`. Al doilea, dacă utilizatorul a indicat un consum mai mare decât cantitatea disponibilă în cămară (greșeală de încadrare, sau introducere ulterioară a unei rețete cu cantitate mai mare), cantitatea este limitată la zero (*clamp*) și rândul se șterge, în loc să primească o valoare negativă. Aceste comportamente de robustețe la marginea fluxului de date au fost validate prin optsprezece cazuri de specificație unitare pentru `cookingSession.complete` la momentul refactorizării pipeline-ului de grame (28 mai 2026).

3.3.3 Pipeline-ul de grame: o euristică, doi consumatori

Cele două fluxuri anterioare — filtrarea progresivă (Secțiunea 3.3.1) și sesiunea de gătit (Secțiunea 3.3.2) — depind de aceeași întrebare practică: *cum convertim cantități între unități eterogene?* Rețeta cere `1 cup` de zahăr și cămara are `500 g`; rețeta cere `2 lemons`, dar cămara are doar `1 piece`. O abordare clasică ar separa fiecare caz (*carve-out*): convertor pentru greutăți, convertor pentru volume, regulă specială pentru bucăți. O astfel de stratificare produce, în timp, ramificații care cresc cu fiecare unitate adăugată.

Soluția adoptată în *Kookio* este o pereche de funcții pure — `toGrams` și `fromGrams` — care normalizează toate cantitățile la grame înainte de orice operație și convertesc înapoi la unitatea de afișare doar la final. Pipeline-ul rezolvă patru clase de unități — greutate, volum, bucăți și gust/ornament —, fiecare cu o regulă de conversie explicită; regulile per clasă, opțiunea `allowDefault` pentru bucățile fără greutate cunoscută și implementarea funcției `toGrams` sunt detaliate în Secțiunea A.2.

Un singur caz merită reținut aici, fiind o decizie de proiectare, nu un simplu detaliu de conversie: unitățile de gust și ornament (`TO_TASTE`, `TO_SERVE`, `GARNISH`) sunt tratate ca `0 g` și excluse din lista de cumpărături. Valoarea `0 g` este o convenție de modelare aleasă conștient, nu o necunoaștere: ea onorează intenția autorului rețetei (cantitatea este lăsată deliberat „după gust"), evită zgomotul în aval (condimentele de bază decrementate la fiecare gătit sau listate la cumpărături nu sunt informație utilă într-o aplicație centrată pe risipa de alimente în vrac) și păstrează o poziție neutră față de aportul dietetic, pe care aplicația nu este în măsură să îl prescrie.

Convenția `0 g` are însă o limitare cunoscută, care decurge direct din ea: ingredientele de bază asezonate „după gust" (sare, ulei, piper) nu sunt urmărite la nivel de cantitate, deci epuizarea lor treptată — prin folosiri mici și repetate — nu este semnalată utilizatorului. Soluția corectă pentru această lacună nu este atribuirea unei valori nenule arbitrare (care ar reintroduce zgomotul descris mai sus și ar produce, în lipsa unei infrastructuri de avertizare, date pe care nimic nu le consumă), ci un mecanism dedicat de urmărire a consumabilelor, bazat pe stare (*în stoc* / *pe terminate* / *epuizat*) marcată de utilizator, nu pe masă estimată. Un astfel de modul — prefigurat de marcajul manual „pe terminate" deja prezent în interfața cămării — este descris ca direcție de evoluție în Secțiunea 4.3.

Decizia structurală pe care o reprezintă acest pipeline depășește detaliul implementării: este o *unificare* între două procese care, anterior, foloseau euristici similare dar separate. La generarea seturilor inițiale de date (*seed*-uri), un pipeline offline de curățare a importat catalogul *TheMealDB* și a estimat greutățile lipsă folosind o tabelă manuală `INGREDIENT_PIECE_GRAMS` construită pentru această etapă. La runtime, decrementarea cămării (Secțiunea 3.3.2) și derivarea listei de cumpărături (Secțiunea 3.3.4) cereau aceleași conversii, dar prin cod scris separat. Prin extragerea funcțiilor `toGrams`/`fromGrams` într-o bibliotecă comună (`libs/shared/utility`) și prin promovarea tabelei manuale la coloana `IngredientCatalog.pieceGrams`, ambele procese consumă acum aceeași sursă de adevăr. Pe planul auditului, o critică a estimării (de exemplu, „densitatea apei este o aproximare slabă pentru uleiuri") se aplică simultan ambilor consumatori; pe planul evoluției, o îmbunătățire (un strat de densități per categorie de ingrediente) se propagă în același mod. Această lecție de proiectare este formulată ca pattern arhitectural generalizabil în Secțiunea 3.2.12.

Perechea `toGrams`/`fromGrams` constituie o aplicare textbook a ascunderii informației [23]: decizia de proiectare schimbabilă — modul concret de conversie între unități eterogene (factori de greutate, aproximarea volumelor prin densitatea apei, valorile `pieceGrams` pentru unitățile de tip bucată) — este izolată în spatele unei interfețe stabile, iar cei doi consumatori (pipeline-ul offline de *seed* și logica de runtime) depind exclusiv de această interfață, nu de implementare. Regula formulată mai sus — o singură euristică defensabilă, doi sau mai mulți consumatori — reformulează criteriul conform căruia interfața unui modul trebuie să dezvăluie cât mai puțin despre mecanismul său intern. Beneficiul este cel descris de Parnas: o critică a estimării (densitatea apei ca aproximare slabă pentru uleiuri) sau o îmbunătățire (un strat de densități per

categorie) se aplică simultan tuturor consumatorilor, fiindcă există un singur loc unde decizia este materializată.

3.3.4 Derivarea listei de cumpărături

Pe ruta `/groceries`, lista de cumpărături apare ca o *diferență de mulțimi pe identitatea ingredientului* între ingredientele cerute de slot-urile planificate și conținutul cămării. Procedura `groceries.list` expandează fiecare slot cu rețetă din planul utilizatorului (toate slot-urile cu `recipeId` nenul) în ingredientele rețetei aferente, scalează cantitățile cu raportul `porții slot / porții rețetă`, agregă pe `ingredientCatalogId` și elimină integral orice ingredient pentru care există deja un rând în cămară — comparația de eliminare se face pe *prezență*, nu pe cantitate. Ingredientele cu unitate *flavour-only* (`TO_TASTE`, `TO_SERVE`, `GARNISH`) se exclud din listă (sare, piper, garnitura nu apar pe lista de cumpărături; `PINCH` este păstrat, având o masă nenulă (0,5 g)).

Reconcilierea în grame nu intervine la filtrarea listei, ci la îmbinarea cu cămara: la final, mutația `groceries.addBoughtToPantry` transferă articolele bifate ca „cumpărate” direct în cămară, contopindu-le cu eventualele rânduri existente pentru același ingredient. Această îmbinare folosește pipeline-ul de grame din Secțiunea 3.3.3 — normalizează cantitățile la grame, le sumează și convertește înapoi la unitatea de stocare a rândului existent (sau a articolului nou, dacă rândul lipsește). Articolele de tip bucată fără `pieceGrams` populată consumă `DEFAULT_PIECE_GRAMS = 100` g prin opțiunea `allowDefault`, soluție de degradare elegantă (*graceful degradation*) preferabilă unui eșec al operației pentru un singur ingredient incomplet.

Starea de „cumpărat” pentru fiecare articol este persistată în Redis (Secțiunea 3.2.7), permițând utilizatorului să continue lista pe *device*-uri diferite în fereastra de expirare a cheii.

3.4 Metodologie și evaluare

3.4.1 Setare experimentală

Toate măsurătorile au fost realizate în mediul *Preview* al Vercel (regiunea `fra1`, Frankfurt), cu CockroachDB Serverless și Redis Cloud în aceeași regiune; stiva rulează Analog peste Nitro, cu SSR la prima cerere și hidratare (hydration) la client. Setul de date provine din seturi deterministe de date de test (*fixture*-uri, `libs/shared/mock/`) și din scripturile de inițializare (*seed*, `libs/prisma-client/seed-data/catalog.json`, `recipes.json`).

Toate procedurile de măsurare sunt versionate în repository-ul lucrării: ruta de benchmark este definită în `apps/web/src/server/routes/api/internal/bench-db.get.ts`, fluxurile de CI sunt în `.github/workflows/`, iar configurația suitei E2E este în `apps/web-e2e/playwright.config.ts`. Fiecare dimensiune se reproduce printr-un script dedicat din

`scripts/`: latența la persistență prin `npm run bench:preview`, acoperirea testelor prin `npm run bench:coverage`, iar performanța front-end — scorurile Lighthouse și payload-ul imaginilor — prin `npm run bench:lighthouse`, respectiv `npm run bench:images`; scripturile care interoghează mediul Vercel necesită credențialele de acces corespunzătoare.

3.4.2 Indicatori și proceduri

Latența la stratul de persistență. Ruta internă `/api/internal/bench-db` execută, la fiecare apel, $N = 100$ interogări succesive de citire către CockroachDB, respectiv Redis, raportând media aritmetică, percentila 50 (p50) și percentila 95 (p95). Distincția rece/cald (*cold*/*warm*) operează la nivelul invocării funcției *serverless*: primul apel plătește pornirea la rece a instanței și a fondului de conexiuni (*connection pool*) către baza de date, în timp ce apelurile ulterioare ating instanța deja încălzită. Se raportează apelul rece separat de mediana apelurilor calde.

Fiabilitatea testelor end-to-end (E2E). Rata de trecere a suitei Playwright pe Chromium, măsurată pe ultimele zece rulări de pe ramura `test`. WebKit trece de asemenea integral, iar Firefox rulează majoritatea testelor, cu excepția câtorva teste de autentificare (marcate `@no-firefox`), excluse din cauza unei curse de sincronizare la nivelul cererilor *fetch* în mediul de dezvoltare local. În CI rulează sistematic doar Chromium; Firefox și WebKit sunt acoperite în execuții manuale înaintea promovărilor în `main`.

Performanță front-end. Se rulează Lighthouse pe pagina de pornire a aplicației publicate la `kookio.app`, în profil desktop și mobil (acesta din urmă cu *throttling* de rețea și de procesor), raportând mediana mai multor rulări pentru scorul de performanță și pentru metricile LCP, CLS și TBT. Separat, se cuantifică reducerea de payload a optimizării de imagini: pentru fiecare dintre cele 594 de imagini ale catalogului se compară fișierul JPEG original cu varianta AVIF livrată de optimizatorul Vercel la lățimea maximă configurată ($w = 960$) — o margine conservatoare, întrucât lățimile responsive mai mici economisesc și mai mult.

Acoperirea testelor unitare. Se raportează acoperirea de linii pe întreg codul-sursă al bibliotecilor (*all-source*: fișierele neacoperite de niciun test sunt numărate ca $0\,\%$, nu omise), agregată din rapoartele Vitest per proiect. Aplicația `web` este exclusă din această cifră, deoarece mediul său de testare expiră sub instrumentarea de acoperire; fluxurile sale de utilizator sunt validate de suita E2E.

Tabelul 2: Sinteza rezultatelor pe cele trei dimensiuni măsurate.

Indicator	**Valoare**
Latența la stratul de persistență ($N = 100$ sonde/apel, mediană pe apelurile calde)	
CockroachDB (cald) — medie	1,79 ms
CockroachDB (cald) — p50	1,71 ms
CockroachDB (cald) — p95	2,07 ms
CockroachDB (rece, *cold start*) — p50 / p95	2,6 / 6,4 ms
Redis (cald) — medie	0,59 ms
Redis (cald) — p50	0,56 ms
Redis (cald) — p95	0,77 ms
Performanță front-end (Lighthouse, mediană)	
Scor desktop	93
Scor mobil (cu *throttling*)	58
LCP desktop / mobil	1,5 / 8,3 s
CLS (desktop / mobil)	0,055 / 0
Reducere payload imagini (594 imagini, $w = 960$)	$-60{,}5\,\%$
Acoperire și testare	
Acoperire linii (cod sursă biblioteci)	$82{,}2\,\%$
Teste unitare (Vitest)	873 / 96 fișiere
Teste E2E (Playwright)	14 / 7 fluxuri
Rată de trecere E2E Chromium	$100\,\%$

3.4.3 Rezultate

Rezultatele sintetizate în Tabelul 2 confirmă, în limitele explicate în Secțiunea 3.4.4, viabilitatea arhitecturii PAN pe cele trei axe de evaluare: latențe milisecundice la stratul de persistență, performanță front-end solidă pe desktop (scor 93, CLS nul) cu un decalaj asumat în profilul mobil cu *throttling*, o reducere de aproximativ $60\,\%$ a payload-ului de imagini pe întregul catalog, o acoperire de $82{,}2\,\%$ a codului bibliotecilor și fiabilitate completă a fluxurilor critice end-to-end pe browserul de referință.

3.4.4 Discuție și limitări

Toate măsurătorile provin din mediul *Preview*, în regiunea `fra1`, fără trafic concurent — așadar nu surprind comportamentul stivei sub încărcare sau în regim multi-regional. Suita E2E rulează doar pe Chromium în CI, iar matricile de testare Firefox și WebKit sunt acoperite manual; o promovare automatizată a celor două ar mări încrederea în compatibilitatea inter-browser (*cross-browser*). În plus, atât CockroachDB, cât și Redis rulează pe planurile gratuite (cu cote de utilizare și ciclu de *auto-suspendare* pentru Redis) și sunt măsurate fără trafic concurent, ceea ce face latențele raportate reprezentative pentru un regim singular, nu pentru producție la scară. Generalizarea acestor rezultate cere teste de încărcare dedicate și măsurători repetate într-un mediu de producție stabil. Mai mult, aceste latențe sunt dominate de furnizorii gestionați pe plan gratuit — timpul de rețea către regiunea `fra1` și limitările cotei gratuite —, astfel încât ele caracterizează infrastructura de deploy, nu *overhead*-ul propriu al stivei PAN. Izolarea acestui overhead ar cere un benchmark local, containerizat (de exemplu cu Docker), fără latența de rețea a furnizorilor de cloud — o direcție de măsurare lăsată lucrărilor viitoare.

Trei limitări specifice noilor măsurători merită semnalate. Întâi, latențele la persistență variază sensibil între seturile de măsurători — mediana p50 pentru CockroachDB a oscilat între aproximativ $1{,}4$ și $2{,}6$ ms —, consecință a planurilor gratuite, a regiunii unice și a încărcării variabile; valoarea raportată în Tabelul 2 este mediana unui set de apeluri calde, nu o constantă. Apoi, scorul Lighthouse pe profilul mobil reflectă *throttling*-ul agresiv aplicat de unealtă (rețea lentă și procesor încetinit de patru ori), nu performanța percepută pe un dispozitiv real — profilul desktop rămâne în zona bună. În fine, acoperirea de $82{,}2\,\%$ privește exclusiv codul bibliotecilor, aplicația `web` fiind exclusă din motivele explicate în Secțiunea 3.4.2.

4 Concluzii

Am propus arhitectura **PAN** și am validat-o implementând *Kookio*, o aplicație web full-stack construită integral în ecosistemul TypeScript pe acest model. Pornind de la preferința pentru TypeScript motivată în Capitolul 2, am tradus această preferință într-o stivă concretă: o singură

aplicație Analog sub `apps/web`, patru grupuri de biblioteci Nx (`feature`, `prisma`, `server`, `shared`) cu topologie aciclică aplicată de `enforce-module-boundaries` la CI, și un flux cap-coadă *Analog* → *tRPC* → *Prisma* descris în Secțiunea 3.1.2.

Adoptarea arhitecturii **PAN** a permis eliminarea straturilor clasice de adaptare între frontend și backend. Procedurile tRPC apelează direct `ctx.prisma` fără un strat de *repository* (Secțiunea 3.2.2), iar schemele Zod consumate de Angular sunt derivate din `schema.prisma` prin `prisma-zod-generator` — o redenumire pe un câmp se propagă la compilare în toți consumatorii. Indexul public `@kookio/prisma-client` este sigur pentru execuție în browser (exportă numai tipuri și scheme Zod), iar instanța `PrismaClient` este izolată sub sub-calea `@kookio/prisma-client/server`, blocată de o regulă ESLint dedicată în codul de frontend.

Combinația dintre **Angular** și **Analog** a livrat SSR pentru toate rutele aplicației, configurat într-un singur loc (`apps/web/vite.config.ts`). Configurația SSR la nivel de aplicație este minimală — `provideFileRouter()`, `provideHttpClient(…)` și `provideClientHydration(withEventReplay())` —, iar disciplina identificată în practică (Secțiunea 3.2.3) este că fiecare import adăugat în `app.config.ts` are un cost amplificat pe traseul critic al fiecărei rute, motiv pentru care componentele Material grele rămân importate de la nivelul paginii care le folosește.

Pe stratul de persistență, **Prisma** peste **CockroachDB** a oferit beneficii observabile direct la nivelul codului. Generarea automată a clientului TypeScript și a schemelor Zod din `schema.prisma` reduce contractul de date la o singură sursă de adevăr; constrângerile compuse declarate ca `@@unique` (`MealSlot` pe `userId–date–slotType`, `PantryItem` pe `userId–ingredientCatalogId`) sunt aplicate la nivelul bazei de date, iar interogările de căutare aferente folosesc forma cu cheie compusă generată automat. Pentru a tolera reluările pe care CockroachDB le poate cere sub izolarea *serializable* implicită, instanța Prisma este extinsă cu o reluare automată la conflict (*retry-on-conflict*) pentru codurile `P2034`, `P2036` și `40001` (plus mesajul *restart transaction*); toate operațiile de scriere (`create`, `update`, `upsert`, `delete` și variantele `*Many`) trec prin acest *wrapper* (înveliș de cod), declarat o singură dată.

Procesul de testare a confirmat valoarea combinării a două niveluri distincte. Suita **Vitest** (orchestrată de `vitest.workspace.ts` la rădăcina repository-ului) acoperă logica izolată prin fișiere de specificații colocate cu codul testat, iar `nx affected -t test` rulează doar specurile atinse de modificarea curentă. Suita **Playwright** de la `apps/web-e2e/` parcurge fluxurile critice (autentificare, *onboarding*, cămară, planificator) prin Chromium și a obținut o rată de trecere de 100 % pe cele zece rulări de referință (Tabelul 2). Două practici s-au consolidat din ciclurile concrete de CI: pre-încălzirea rutelor în `global-setup.ts` înainte ca primul proces de lucru (*worker*) să pornească, și `waitUntil: 'commit'` pentru rutele protejate, ca răspuns la eroarea `ERR_ABORTED` produsă de redirecționările Angular în timpul hidratării.

Din perspectiva obiectivelor declarate în Capitolul 1, am livrat un produs minim viabil (MVP) care acoperă fluxurile centrale ale aplicației: gestiunea cămării, planificarea zilnică a meselor, derivarea listei de cumpărături și *onboarding*-ul utilizatorului. Structura de monorepo pe felii verticale (`libs/feature/auth`, `onboarding`, `pantry`, `plan`, `recipes`…), în care `libs/feature/X` nu poate importa din `libs/feature/Y` — regulă verificată la CI prin `enforce-module-boundaries` —, permite extinderea cu fluxuri noi ca biblioteci adiționale, fără modificări structurale ale aplicației existente.

În ansamblu, *Kookio* arată că arhitectura **PAN** poate funcționa coerent pentru o aplicație web full-stack reală: latențele măsurate pe ruta `/api/internal/bench-db` (mediană pe apeluri calde 1,71 ms pentru CockroachDB, 0,56 ms pentru Redis — Tabelul 2), performanța front-end (scor Lighthouse 93 pe desktop și o reducere de circa 60 % a payload-ului de imagini pe întregul catalog), acoperirea de 73,2 % a codului bibliotecilor și rata de trecere de 100 % a suitei E2E pe Chromium susțin acest model în limitele explicate în Secțiunea 3.4.4 — mediul *Preview*, regiune unică `fra1`, planuri gratuite pentru CockroachDB și Redis, matricea de testare inter-browser (*cross-browser*) acoperită manual și absența traficului concurent, latențele raportate reflectând un regim secvențial, nu performanța la scară.

Contribuția principală a prezentei lucrări este propunerea și validarea empirică a arhitecturii **PAN** ca model coerent pentru aplicații web full-stack în ecosistemul Angular/TypeScript, demonstrată printr-un studiu de caz funcțional în domeniul reducerii risipei alimentare la nivel casnic.

4.1 Decizii arhitecturale

Pe parcursul implementării, mai multe decizii inițiale au fost rafinate, eliminate sau reconfigurate ca răspuns la frecușul de implementare. Cele nouă decizii arhitecturale documentate în repository-ul proiectului se grupează în trei categorii distincte, prezentate sintetic în Tabelul 3: patru *simplificări* (eliminarea unor sisteme construite, dar fără consumator real), trei *unificări* (pattern-uri descoperite din frecuș și codificate ca regulă, prezentate detaliat în Secțiunea 3.2.12) și două *decizii de design* (alegeri de interfață revizuite post-implementare).

Pe planul lucrării de licență, fiecare decizie reprezintă o *ajustare a invariantului* pe care arhitectura PAN l-a expus la întâlnirea cu un caz de utilizare concret. Simplificările (categoriile 1–4) arată că o stivă coerentă tipologic permite identificarea rapidă a codului mort prin căutare pe *tipuri de consumatori*, nu doar pe căi de import: dacă o procedură tRPC, un *guard* sau un model Prisma nu este referit nicăieri în frontend, eliminarea sa este sigură la nivel de compilare. Unificările (categoriile 5–7) ilustrează că arhitectura PAN expune frecușuri care nu sunt vizibile în prezentarea teoretică a stivei (Secțiunea 2.3), dar care apar din interacțiunea dintre componente la rulare. Deciziile de design (categoriile 8–9) sunt independente de PAN ca atare, dar arată că

Tabelul 3: Cele nouă decizii arhitecturale documentate, organizate pe categorii.

Categorie	Decizie	Lecție generalizabilă
Simplificări — sistem construit, zero consumator real, eliminare		
1	Eliminarea autentificării biometrice (*WebAuthn*)	Sistemul a fost construit ca răspuns la o cerință anticipată, nu observată; nicio interfață nu îl consuma efectiv.
2	Eliminarea infrastructurii de roluri (control de acces bazat pe roluri)	Singurul consumator era o pagină stub pentru administrator; eliminarea a redus enum-ul `Role`, coloana `User.role` și *builder*-ul `roleProcedure`.
3	Aplatizarea agregatului `MealPlan`	Aggregate-ul intermediar nu avea consumator real; UI-ul lucra direct cu `MealSlot` pe `(userId, date, slotType)`. Eliminarea a redus cu unul numărul de modele din schemă.
4	Eliminarea grupului de biblioteci `core/`	Subsistemul de *feature flags* și preocupările de configurare/securitate erau complet cablate, dar fără consumator (router-ul nu era montat, *guard*-ul nu păzea nicio rută); eliminarea a șters bibliotecile `core/config` și `core/security` plus tokenul `APP_TRPC`, iar `core/logging` a fost absorbit în `server/logger` — taxonomia a trecut de la cinci la patru grupuri.
Unificări — frecuș de implementare, lecție codificată (Secțiunea 3.2.12)		
5	Încărcătoarele SSR cer hidratare client-side a datelor autentificate	*Bearer token*-ul Firebase nu se propagă prin `fetch` pe navigare client-side; loader-ele apelează doar proceduri publice.
6	Rutele numite semantic, nu prin coduri numerice	Router-ul bazat pe fișiere al Analog cade silentios pe segmente *all-numeric*; soluția: `/server-error` în loc de `/500`.
7	O singură euristică pentru estimarea cantităților, doi consumatori	Pipeline-ul de curățare offline și logica de runtime consumă acum aceeași pereche `toGrams`/`fromGrams` (Secțiunea 3.3.3).
Decizii de design — propunere inițială, validare la utilizare ulterioară		
8	Buton *Cook now* mutat din UI-componenta *FAB* în topbar	Componenta *FAB* (*floating action button*) ocupa spațiu fără a oferi un avantaj de vizibilitate față de un buton în topbar.
9	Widget-ul *Quick actions* eliminat din pagina principală	Acțiunile rapide propuse erau redundante cu bara de navigare de jos plus butonul *Cook now*; eliminarea a redus zgomotul vizual al paginii principale.

disciplina structurală a stivei nu împiedică revizuirea propunerilor de interfață — componentele eliminate erau organizate în biblioteci proprii și au putut fi scoase fără a atinge restul aplicației.

Tonul reflexiv al acestor decizii — rafinate sau eliminate la întâlnirea cu un caz concret — are un precedent direct în articolul fondator al ascunderii informației: în secțiunea „*Improvement in Circular Shift Module*", Parnas își analizează propriul modul și constată că interfața dezvăluia mai multă informație decât necesar (impunea o ordine a shift-urilor), restrângând inutil clasa de implementări posibile — eroare pe care o clasifică explicit drept eroare de proiectare [23]. Documentarea propriilor decizii revizuite nu este, așadar, un semn de imaturitate a proiectului, ci o practică prezentă chiar în textul care a introdus principiul.

4.2 Limitări și compromisuri ale arhitecturii

Adoptarea PAN ca arhitectură nu este lipsită de compromisuri. Această subsecțiune le listează explicit, pentru ca un cititor care evaluează PAN pentru un proiect propriu să poată face o alegere informată:

- **Curba de adoptare.** PAN cere cunoștințe consolidate de Angular și de TypeScript avansat (*generics*, *conditional types*, *mapped types*). Pentru o echipă fără experiență Angular, învățarea PAN are un cost semnificativ mai mare decât adoptarea unui Next.js convențional.
- **Maturitate Analog.** Analog este, la momentul scrierii, un proiect cu o comunitate de câteva mii de utilizatori și fără adopție majoră în mediul corporativ. Riscul ca anumite probleme de producție să nu aibă răspuns documentat este real. Mitigarea folosită în prezenta lucrare a fost selectarea componentelor mature individual (Angular, Nitro) și acceptarea explicită a riscului pentru stratul de orchestrare Analog.
- **Cuplaj la TypeScript.** tRPC presupune că ambele capete ale comunicării sunt TypeScript. PAN nu este o opțiune viabilă pentru aplicații care trebuie să expună un API public consumat de clienți în alte limbaje (Java, Go, Python). În astfel de cazuri, GraphQL sau OpenAPI rămân alegeri mai potrivite.
- **Dependență moderată de furnizor.** Deși PAN poate fi publicat pe orice infrastructură compatibilă Node.js (Nitro suportă Vercel, Netlify, AWS, Cloudflare, Bun, Deno), integrarea cu Vercel este cea mai bine bătătorită. Migrarea către un alt furnizor este posibilă, dar nu este lipsită de efort.
- **CockroachDB la scară mică.** Pentru aplicații sub 100 000 cereri pe zi, complexitatea operațională a CockroachDB (algoritmul de consens Raft, izolarea *serializable* implicită care necesită logică de reluare) este disproporționată față de PostgreSQL standard. PAN cu PostgreSQL clasic ar fi o variantă mai simplă pentru aplicații care nu necesită scalare orizontală reală.

- **Resurse de învățare limitate.** Conținutul comunitar (Stack Overflow, articole de blog, cursuri) pentru PAN ca atare nu există — există doar pentru componentele individuale. Un dezvoltator care întâmpină o problemă specifică integrării Analog + tRPC + Prisma + Nx trebuie să își construiască soluția pornind de la primele principii.

- **Câștiguri cedate prin renunțarea la distribuție.** Aceeași analiză [32] identifică drept câștiguri operaționale majore ale microserviciilor scalabilitatea orizontală independentă (câștigul #1, 31 de studii), *deploy*-ul independent al componentelor și libertatea de a alege un *stack* tehnologic distinct pentru fiecare serviciu. PAN cedează explicit primele două: aplicația se publică unitar, iar scalarea reală sub trafic concurent nu a fost caracterizată (Secțiunea 3.4.4). A treia este chiar negativul unuia dintre principiile sale fondatoare — uniformitatea TypeScript de la capăt la capăt (Secțiunea 2.2) este aleasă tocmai în detrimentul libertății tehnologice per componentă. Pentru un proiect care necesită scalare independentă a unor subsisteme cu profiluri de încărcare divergente, sau care trebuie să integreze servicii scrise în limbaje diferite, aceste compromisuri ar înclina balanța înapoi către o arhitectură distribuită.

PAN nu este, prin urmare, o arhitectură universal aplicabilă. Este o arhitectură pentru echipe mici cu cunoștințe Angular consolidate care construiesc aplicații TypeScript moderne, monolitice (în sensul publicării unitare) și centrate pe date. În acest spațiu de soluții, PAN oferă o coerență tehnologică și o disciplină structurală pe care alternativele bazate pe React le ating doar prin alegeri și convenții *ad-hoc*.

4.3 Direcții viitoare

Direcțiile naturale de evoluție pornesc de la limitele identificate în Secțiunea 3.4.4 și de la compromisurile arhitecturale discutate în Secțiunea 4.2. La nivelul infrastructurii, promovarea CockroachDB-ului și a Redis-ului pe planuri plătite, alături de activarea unei a doua regiuni dincolo de `fra1` (CockroachDB suportă nativ replicarea cross-zone, activabilă fără modificări de cod aplicativ), ar permite caracterizarea stivei sub trafic concurent. În domeniul testării, includerea sistematică a matricilor de testare Firefox și WebKit în CI ar consolida încrederea în compatibilitatea inter-browser (*cross-browser*) dincolo de Chromium. Pe planul funcționalității, infrastructura Redis existentă (folosită deocamdată pentru marcarea articolelor cumpărate din lista de cumpărături și pentru limitarea ratei pe ruta de benchmark — Secțiunea 3.2.7) poate fi extinsă cu un strat de *caching* aplicativ pentru paginile publice. Recomandarea rețetelor pe baza inventarului din cămară este deja acoperită de filtrarea progresivă (Secțiunea 3.3.1), iar valorificarea preferințelor din *onboarding* este parțial implementată — filtrarea opțională după categoriile și ariile culinare preferate. Restricțiile dietetice sunt deja colectate și persistate în fluxul de *onboarding* (un pas dedicat le scrie în `UserDietaryRestriction`), însă nu participă

încă la recomandare: ar rămâne de adăugat doar etichetarea rețetelor cu restricțiile pe care le satisfac — o relație `Recipe–DietaryRestriction` absentă deocamdată din schemă, derivabilă din ingredientele fiecărei rețete — după care filtrul s-ar racorda la același mecanism opțional folosit pentru categorii și arii. Tot la acuratețea potrivirii contribuie și normalizarea catalogului de ingrediente: potrivirea dintre rețetă și cămară se face pe identitatea ingredientului din catalog, însă o parte dintre ingredientele rețetelor importate folosesc denumiri generice (de exemplu „flour” sau „potatoes”) care nu au un corespondent direct în catalog — acesta conține doar variante calificate, precum *All purpose flour* sau *Plain Flour* — astfel încât rețetele care le cer nu pot atinge nivelul *Alright* pornind de la conținutul cămării. Extinderea tabelei de echivalențe (*aliasuri*) aplicate la importul datelor (Secțiunea 3.2.5) ar reduce aceste nepotriviri și ar mări proporția de rețete complet acoperite. Tot pe această axă, un modul de urmărire a consumabilelor ar acoperi limitarea identificată în Secțiunea 3.3.3: ingredientele de bază asezonate „după gust” (sare, ulei, piper), care în prezent nu sunt cuantificate, ar fi urmărite la nivel de stare (*în stoc* / *pe terminate* / *epuizat*) marcată de utilizator, iar articolele semnalate ca epuizate ar alimenta automat lista de cumpărături, fără a recurge la estimări de masă pentru cantități lăsate intenționat la latitudinea bucătarului.

Dincolo de direcțiile specifice produsului *Kookio*, arhitectura PAN însăși rămâne extensibilă pe mai multe axe neexplorate în studiul de caz:

- **Comunicare în timp real.** tRPC suportă *subscriptions* prin WebSocket sau Server-Sent Events; o ramură naturală de evoluție este integrarea cu un broker de mesaje (Redis Pub/Sub, NATS) pentru notificări către clienți autentificați.
- **Funcții la nivelul rețelei de margine (*edge functions*).** Nitro permite publicare parțială la nivelul *edge* (Vercel Edge Functions, Cloudflare Workers). Procedurile tRPC care nu necesită Prisma pot fi mutate la *edge* pentru reducerea latenței geografice.
- **Aplicații mobile.** Stiva Angular are suport pentru aplicații mobile native prin Capacitor sau NativeScript; bibliotecile `shared` și `prisma` ale unui proiect PAN sunt direct reutilizabile între o aplicație web și una mobilă.
- **Internaționalizare.** Angular i18n combinat cu rutare multilingvă Analog (rute parametrizate prin prefix de limbă) acoperă cazul aplicațiilor multilingve fără modificări arhitecturale.

Topologia aciclică a monorepo-ului face ca aceste extensii să poată fi realizate incremental: o nouă bibliotecă în `libs/feature/` cu propriul dispecer de rute tRPC se adaugă fără a atinge restul aplicației.

Anexa A Demonstrarea aplicației *Kookio*

Această anexă documentează fluxul funcțional principal al aplicației *Kookio*, ca dovadă a faptului că produs minim viabil-ul (MVP) descris în Secțiunea 3 este operațional de la un capăt la altul. Capturile de ecran surprind parcursul real al unui utilizator — de la inventarul cămării, prin planificarea meselor și lista de cumpărături, până la sesiunea de gătit care decrementează stocul. Secțiunea finală relochează aici detaliul de implementare al pipeline-ului de conversie a unităților, păstrat în corpul lucrării doar la nivelul deciziei de proiectare (Secțiunea 3.3.3).

A.1 Fluxul principal: de la configurare la gătit

Parcursul demonstrat acoperă lanțul de valoare al aplicației, de la configurarea inițială a contului până la actualizarea automată a stocului după gătit: utilizatorul își stabilește preferințele, declară ce are în cămară, vede ce poate găti, planifică mesele, derivă lista de cumpărături pentru ce lipsește și, după gătit, cămara se decrementează.

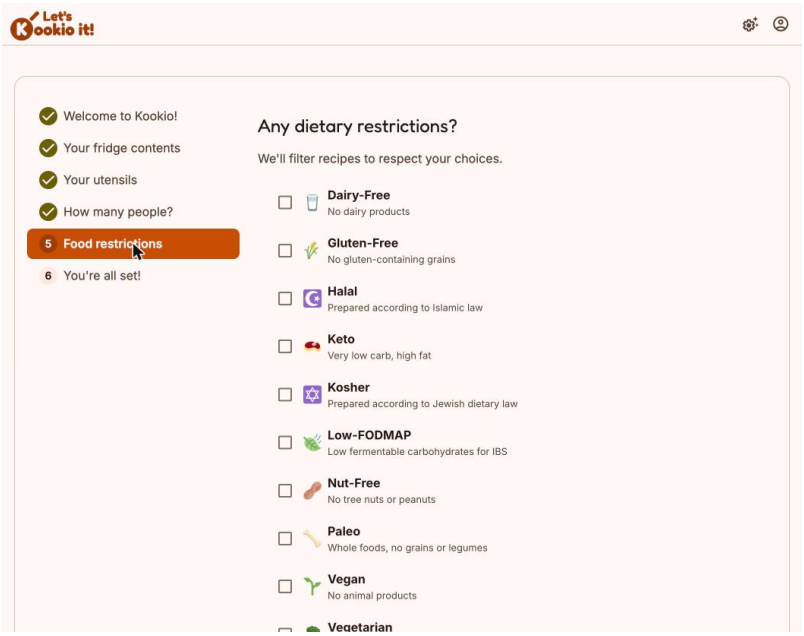


Figura 5: Onboarding (`/onboarding`): pașii de configurare inițială — preferințe culinare (categorii și arii), restricții dietetice și ustensile disponibile — persistați la finalizare și folosiți ulterior în filtrarea rețetelor.

A.2 Detaliul pipeline-ului de conversie a unităților

Corpul lucrării (Secțiunea 3.3.3) reține pipeline-ul de conversie a unităților la nivelul deciziei de proiectare — perechea de funcții pure `toGrams`/`fromGrams` ca aplicare a ascunderii informației.

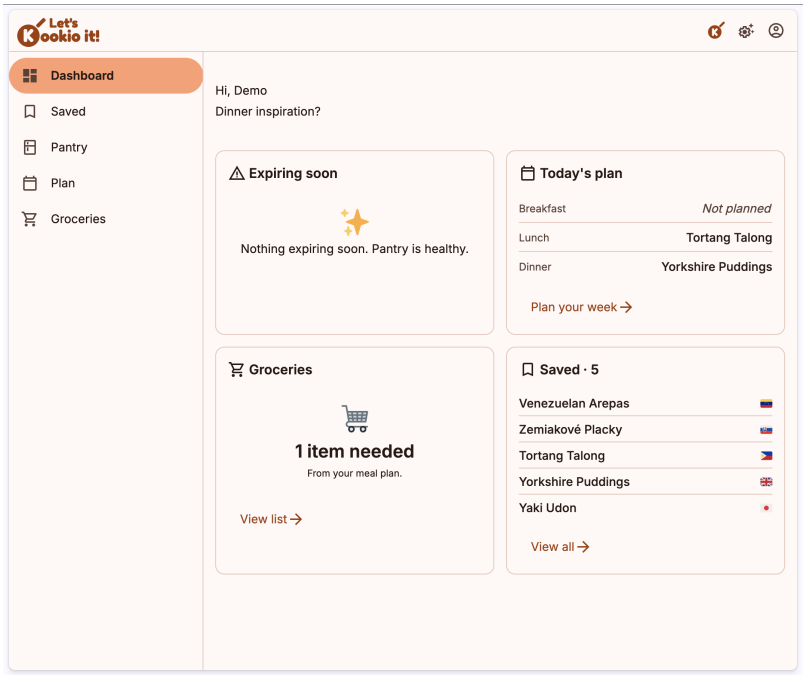


Figura 6: Panoul principal (`/dashboard`): punctul de intrare după autentificare, cu acces către fluxurile aplicației și un rezumat al stării curente.

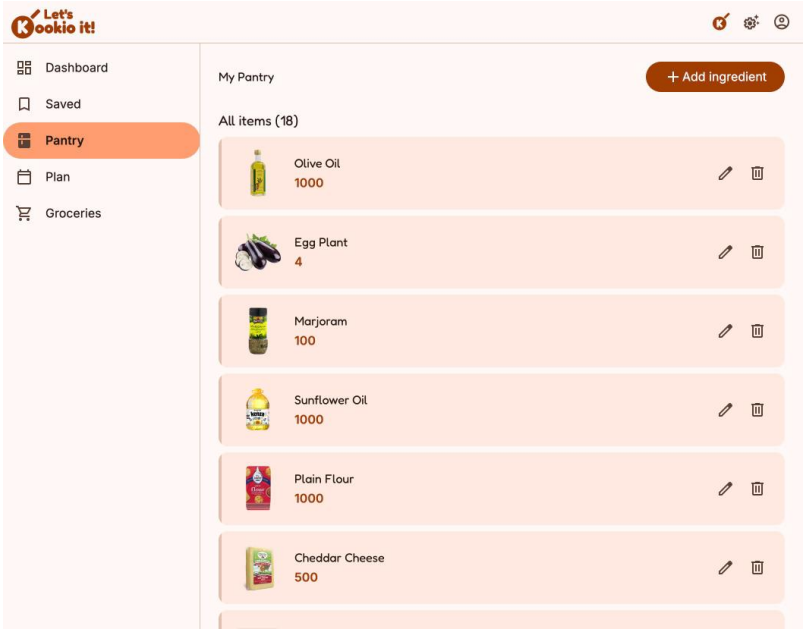


Figura 7: Cămara (`/pantry`): inventarul utilizatorului, cu fiecare ingredient stocat în unitatea sa de afișare. Acesta este punctul de pornire al fluxului — sursa de adevăr pentru ce este disponibil acasă.

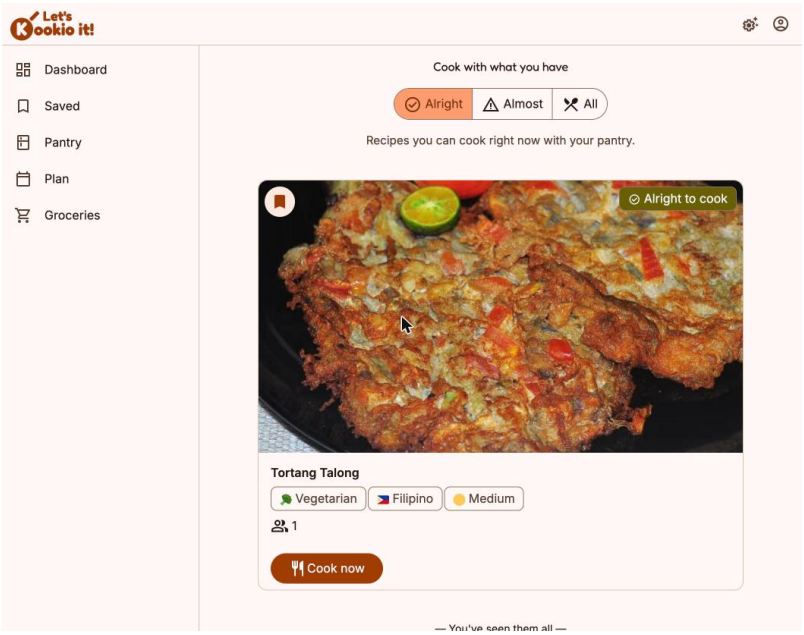


Figura 8: Filtrarea progresivă (`/recipes/match`): catalogul partiționat după gradul de acoperire din cămară (Secțiunea 3.3.1). Nivelul *Alright* listează rețetele complet acoperite, *Almost* pe cele cărora le lipsesc unu–trei ingrediente.

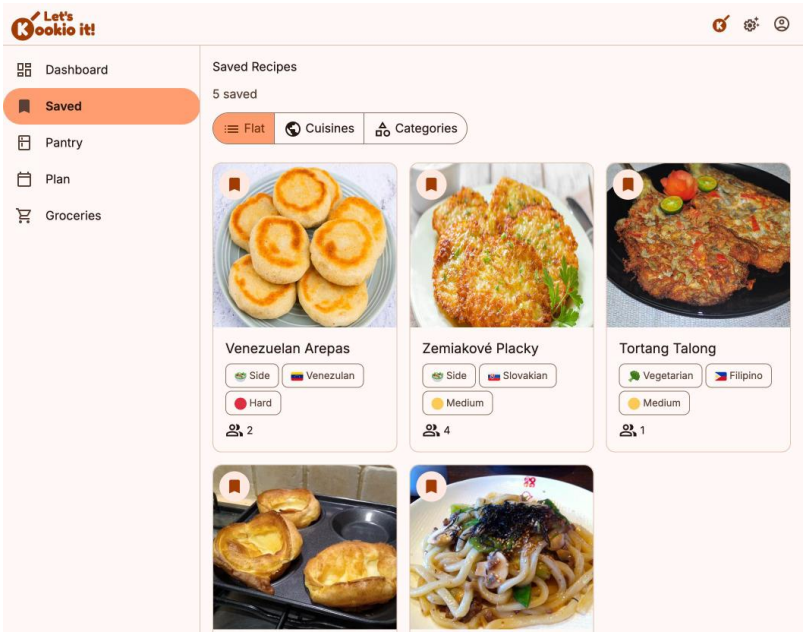


Figura 9: Rețete salvate (`/recipes/saved`): colecția de rețete marcate de utilizator pentru acces ulterior, independentă de gradul de acoperire din cămară.

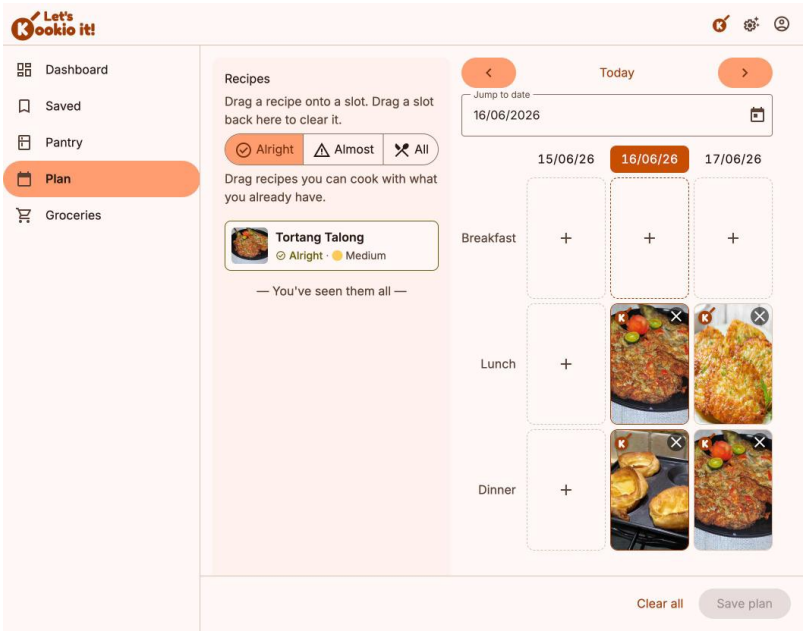


Figura 10: Planificatorul (`/plan`): rețetele se atribuie pe slot-uri `(dată, tip de masă)`, modelate ca `MealSlot` cu constrângerea `@@unique` descrisă în Secțiunea 3.2.4.

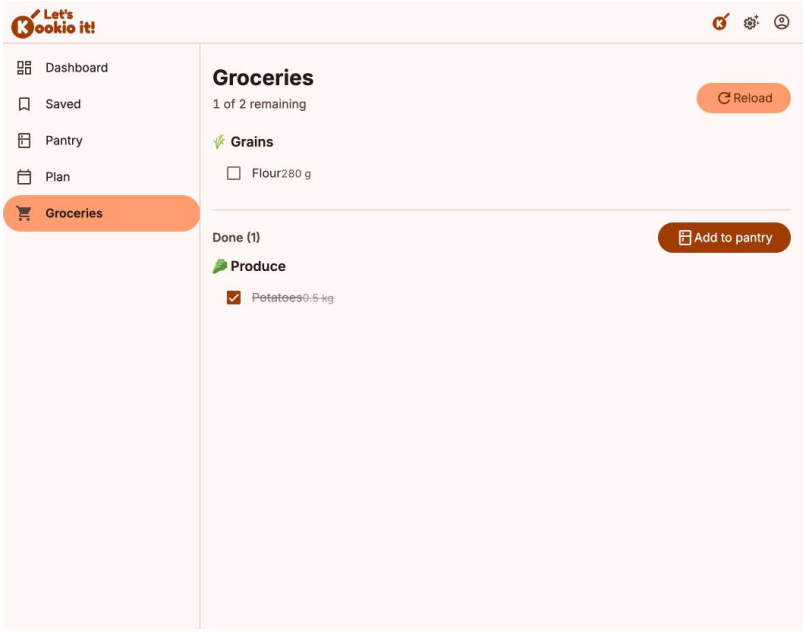


Figura 11: Lista de cumpărături (`/groceries`): diferența de mulțimi între ingredientele slot-urilor planificate și conținutul cămării (Secțiunea 3.3.4). Articolele bifate ca „cumpărate” se pot îmbina înapoi în cămară.

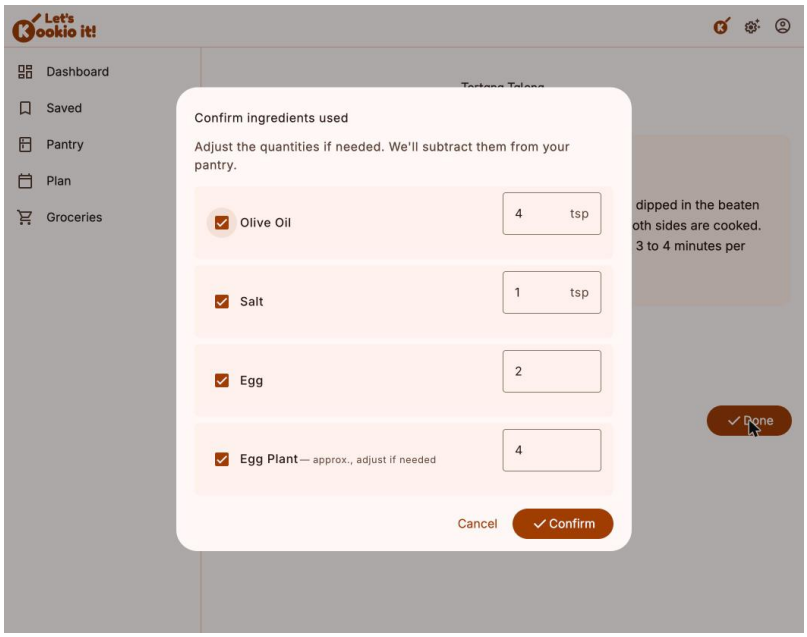


Figura 12: Sesiunea de gătit (`/cook/:recipeId`): la final, dialogul propune cantitățile consumate pre-completate, iar confirmarea declanșează decrementarea tranzacțională a cămării (Secțiunea 3.3.2).

Detaliul concret al regulilor de conversie este relocat aici, fiind un detaliu de implementare specific aplicației, util ca referință, dar nu necesar argumentului arhitectural.

Pipeline-ul rezolvă patru clase de unități, fiecare cu o regulă explicită:

- **Greutate** (`G`, `KG`, `OZ`, `LB`): conversie directă prin factori constanți.
- **Volum** (`ML`, `L`, `TSP`, `TBSP`, `CUP`): conversie prin densitatea apei (`1 ml` $\approx$ `1 g`), aproximare acceptabilă pentru ingredientele uzuale din gătitul casnic.
- **Bucăți** (`PIECE`, `SLICE`, `CLOVE`, `BUNCH`, `OTHER`): conversie prin `IngredientCatalog.pieceGrams`, o coloană opțională (`Float?`) populată cu 32 de valori (`egg: 60`, `garlic clove: 5`, `lemon: 100` etc.).
- **Gust și ornament** (`TO_TASTE`, `TO_SERVE`, `GARNISH`): tratate ca `0 g` și excluse din lista de cumpărături. Distincția față de `PINCH` nu este una de mărime, ci de natură: `PINCH` este o cantitate specificată de autorul rețetei (o ciupitură), pe când unitățile de gust și ornament exprimă o instrucțiune — „după gust”, „pentru servire” — a cărei cantitate este lăsată deliberat la latitudinea celui care gătește. Valoarea `0 g` este, prin urmare, o convenție de modelare aleasă conștient, nu o necunoaștere: ea onorează intenția autorului (a atribui un gram-echivalent ar suprascrie o decizie lăsată explicit deschisă), evită zgomotul în aval (o jumătate de gram de sare decrementată la fiecare gătit, sau condimentele de bază listate la cumpărături, nu sunt informație utilă într-o aplicație centrată pe risipa de alimente în vrac) și păstrează o poziție neutră față de aportul dietetic, pe care aplicația nu este în măsură

să îl prescrie. Pentru PINCH, în schimb, o convenție stabilește 0,5 g per bucată — masă mică, dar reală, suficientă pentru a permite ștergerea rândurilor sub-ciupitură din cămară.

Pentru ingredientele de tip bucată fără valoare pieceGrams populată, funcția întoarce null, semnalând apelantului că nu poate produce o estimare. Un parametru opțional allowDefault schimbă acest comportament: când este activat explicit, funcția returnează DEFAULT_PIECE_GRAMS = 100 g per bucată. Valoarea de 100 g este o aproximare grosieră, aleasă ca ordin de mărime plauzibil pentru o bucată oarecare, nu o medie măsurată empiric; rolul ei este de plasă de siguranță, nu de estimare exactă — de aceea ea nu este aplicată implicit, ci doar la cererea explicită a apelantului. Această opțiune protejează apelanții care preferă eroarea explicită (sesiunea de gătit, unde un consum nefondat ar corupe inventarul) de a accepta tăcut estimări nesigure, dar permite consumatorilor care preferă degradarea elegantă (lista de cumpărături, unde o estimare aproximativă este preferabilă unui eșec total) să continue procesarea.

Lista 1 prezintă funcția toGrams. Implementarea completă, alături de inversa fromGrams, se află în libs/shared/utility/src/lib/labeling/measure-unit-labels.ts.

Lista 1: Funcția toGrams — normalizarea cantităților la grame.

```
export const toGrams = (
  amount: number,
  unit: MeasureUnit,
  pieceGrams: number | null = null,
  opts: GramsConversionOptions = {},
): number | null => {
  if (FLAVOUR_ONLY_UNITS.has(unit)) return 0;

  const fixed = UNIT_TO_GRAMS_FIXED[unit];
  if (fixed !== undefined) return amount * fixed;

  // Piece-class: PIECE, SLICE, CLOVE, BUNCH, OTHER.
  const piece =
    pieceGrams !== null && pieceGrams > 0
      ? pieceGrams
      : opts.allowDefault
        ? DEFAULT_PIECE_GRAMS
        : null;
  if (piece === null) return null;
  return amount * piece;
};
```

Anexa B Modelul de date

Schema completă a bazei de date — cele 20 de modele Prisma — este redată în Figura 13 ca diagramă entitate-relație. Diagrama este generată automat din `schema.prisma` (cu *prisma-dbml-generator*) și randată cu *dbdocs.io*, rămânând astfel sincronizată cu modelul de date efectiv. Cheile primare sunt marcate cu simbolul cheii, cheile externe cu simbolul de legătură, iar conectorii dintre tabele redau relațiile dintre entități; tabelele de joncțiune (de exemplu `UserDietaryRestriction`, `RecipeCollectionEntry`) sunt așezate lângă entitățile pe care le leagă.

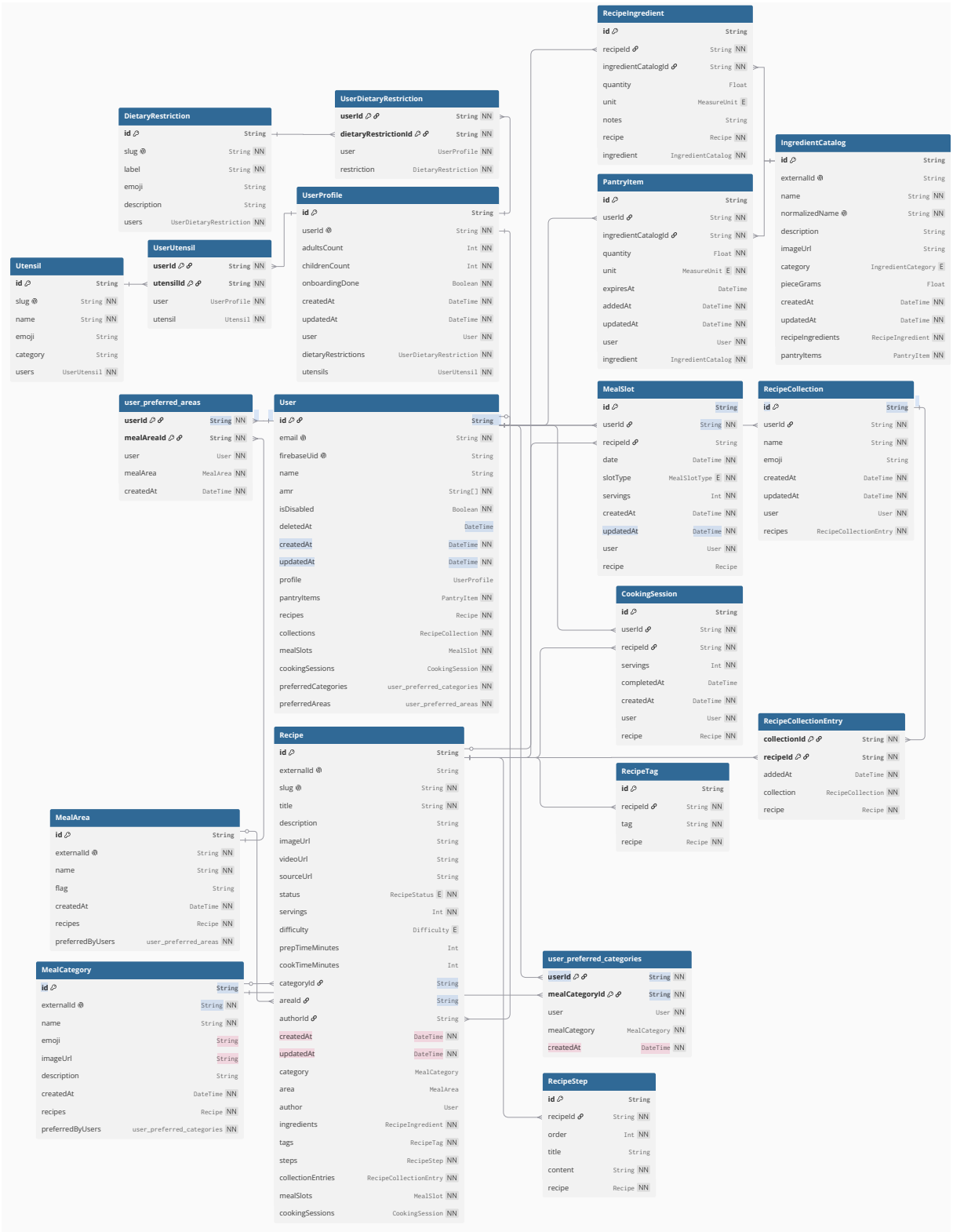


Figura 13: Diagrama entitate-relație a schemei de date: cele 20 de modele și relațiile dintre ele, generate din `schema.prisma` și publicate pe *dbdocs.io*. Versiunea interactivă, navigabilă: https//dbdocs.io/dinea.eduard/kookio-db (parolă de acces: `FMI_2026`).

Glosar de termeni

Analog meta-framework full-stack pentru Angular, cu suport SSR și rutare pe fișiere; folosit pentru aplicații web

Angular platformă și framework pentru aplicații web (SPA/SSR)

Lighthouse unealtă Google de audit automat al performanței paginilor web, care raportează metrici *Core Web Vitals* (LCP, CLS, FCP, TBT)

Nitro runtime universal (UnJS) pentru rute server și randare pe server (SSR)

Nx sistem de build și orchestrare pentru monorepo-uri JavaScript/TypeScript

Prisma bibliotecă ORM pentru TypeScript/Node.js

Zod bibliotecă de validare pe bază de schemă pentru TypeScript/JavaScript

10

CockroachDB sistem de gestiune a bazelor de date distribuit (NewSQL), compatibil la nivel de protocol cu PostgreSQL, cu replicare și partiționare automate pentru reziliență și scalare orizontală

composite action tip de acțiune reutilizabilă în GitHub Actions care grupează mai mulți pași (`run`/`uses`) într-o singură acțiune; se distribuie ca director cu fișier `action.yml`, acceptă *inputs*, *outputs* și *secrets* și poate fi apelată dintr-un workflow prin `uses:`. Utilă pentru a elimina duplicarea (DRY) în pipeline-urile CI/CD; limitări uzuale: nu rulează în containere proprii și nu pot defini *services*.

hidratare (hydration) procesul prin care codul din client atașează comportament interactiv la *markup*-ul generat prin SSR; permite trecerea de la randare inițială pe server la interacțiuni CSR fără reîncărcarea paginii

Kookio aplicație web full-stack folosită ca studiu de caz în această lucrare pentru gestiunea cămării, planificarea zilnică a meselor și derivarea automată a listei de cumpărături, construită pe arhitectura PAN

jurnalizare (logging) colectarea și înregistrarea evenimentelor aplicației (mesaje structurate pe niveluri, corelare prin identificatori, export către sisteme de agregare) pentru depanare și observabilitate

meta-framework framework care extinde un framework de bază pentru capabilități full-stack (ex. rutare pe fișiere, SSR/SSG, integrare cu tRPC). În această lucrare, Analog extinde Angular peste runtime-ul Nitro.

monorepo strategie de organizare a codului în care mai multe aplicații și biblioteci coexistă într-un singur depozit, cu orchestrare unitară a build-urilor, testelor și CI/CD; în contextul lucrării este gestionat cu Nx

Next.js framework React orientat pe randare hibridă (SSR/SSG), frecvent asociat cu JAMstack

Pino bibliotecă de logare pentru Node.js, orientată pe performanță (JSON logs, output low-overhead)

Playwright bibliotecă pentru automatizarea browserelor și testare E2E, cu suport pentru Chromium, Firefox și WebKit, rulare headless/GUI și unelte precum trace viewer

PostgreSQL sistem de gestiune a bazelor de date relaționale open-source, cu suport extins pentru standardul SQL, tranzacții ACID și extensibilitate prin tipuri și funcții definite de utilizator

Prisma Client client `TypeScript` pentru baza de date, generat automat de Prisma din fișierul `schema.prisma`; oferă un API tipizat pentru operații SQL (CRUD, filtre, relații, tranzacții) asupra modelelor definite. Începând cu Prisma 7, clientul rulează integral în JavaScript (fără motor nativ în Rust): conexiunea trece printr-un *driver adapter* (`@prisma/adapter-pg`), iar regenerarea cu `npx prisma generate` produce doar cod tipizat, independent de platforma de execuție.

randare procesul prin care o aplicație generează HTML (și actualizări ale interfeței) din cod și date. În web-ul modern poate avea loc pe server (SSR), la build-time prin SSG/prerender, sau în browser (CSR); de regulă, este urmată de hidratare (hydration) pentru interactivitate. În Analog, strategia de randare poate fi aleasă per rută.

Redis magazin de date *in-memory* tip cheie–valoare, cu structuri avansate (liste, seturi, hash-uri, mulțimi ordonate, *streams*) și latență foarte mică; folosit frecvent pentru caching, stocare sesiuni, cozi/mesagerie (Pub/Sub, Streams), rate-limiting și *leaderboards*. Suportă persistență opțională (RDB/AOF), replicare, *cluster* pentru partiționare și, de regulă, completează o bază SQL principală în arhitecturi microservicii și *serverless*.

Sentry platformă de monitorizare a erorilor în producție, care agregă, grupează și alertează asupra excepțiilor captate din client și din server

TypeScript superset tipizat al limbajului JavaScript care introduce tipuri statice și se transpilează la JavaScript

USES relație definită de Parnas (1972) între module: un modul A se află în relația *USES* cu modulul B dacă funcționarea corectă a lui A depinde de prezența unei versiuni corecte a lui B. Parnas o scrie cu majuscule ca termen formal și cere ca această relație să fie o ordine parțială fără bucle (aciclică), proprietate care permite construirea de subseturi funcționale ale sistemului prin „retezarea” nivelurilor superioare

Vercel platformă cloud pentru build, preview și deploy de aplicații web, cu integrare CI/CD, variabile de mediu per mediu și suport pentru SSR, funcții *serverless* și *edge functions*

Vite instrument de build și server de dezvoltare pentru aplicații web moderne, bazat pe module ECMAScript, cu pornire foarte rapidă și *hot module replacement* (HMR); stă la baza Vitest și este folosit de Analog pentru SSR și build

Vitest framework de testare rapid pentru ecosistemul Vite, orientat spre teste unitare și de integrare, cu API în mare parte compatibil Jest și suport nativ pentru TypeScript

web ecosistemul de resurse *hipertext* accesibile prin Internet (World Wide Web); în lucrare, „aplicație web” desemnează aplicație accesată din browser, cu randare SSR/SSG/CSR

workflow configurație declarativă care descrie o succesiune de pași automatizați (build, testare, lint, deploy) declanșați de evenimente (ex.: push, pull request, programare). În ecosisteme precum GitHub Actions și GitLab CI, un workflow este compus din job-uri și pași, poate folosi matrici, cache, artefacte, secrete/variabile și reguli de concurență. În acest proiect, workflow-urile orchestrează CI/CD și aliniază execuția la țintele Nx.

Lista de acronime

AVIF format de imagine bazat pe codecul AV1

CI integrare continuă

CI/CD integrare continuă și livrare/implementare continuă

CLS deplasare cumulativă a aspectului (*Cumulative Layout Shift*)

CSP Content-Security-Policy

CSR randare pe client

DSN Data Source Name

DTO obiect de transfer de date

E2E testare cap-coadă

FCP prima randare de conținut (*First Contentful Paint*)

HSTS HTTP Strict-Transport-Security

LCP cea mai mare randare de conținut (*Largest Contentful Paint*)

MEAN MongoDB–Express–Angular–Node.js

MVP produs minim viabil

ORM mapare obiect–relațională

p50 percentila 50 (mediană)

p95 percentila 95

PAN Prisma–Analog–Nx

PR cerere de integrare (pull request)

SEO optimizare pentru motoarele de căutare

SPA aplicație web de o singură pagină (single-page application)

SSG generare statică de pagini

SSR randare pe server

TBT timp total de blocare (*Total Blocking Time*)

tRPC apeluri de proceduri la distanță tipizate pentru TypeScript

UI interfață de utilizator

Referințe

[1] Autorii Pino, *Pino — Logger pentru Node.js*, Documentația Pino, Documentație oficială Pino; citată pentru logarea centralizată utilizată atât în backend, cât și în frontend., 2026, URL: https://getpino.io/#/docs/transports (accesat în 18.5.2026).

[2] Autorii Sentry, *Sentry — Monitorizarea erorilor pentru aplicații*, Documentația Sentry, Documentație oficială Sentry; citată pentru monitorizarea erorilor în producție, integrată simetric pe client și pe server., 2026, URL: https://docs.sentry.io/ (accesat în 13.6.2026).

[3] Autorii Zod, *Documentația Zod*, Documentația Zod, Documentație oficială Zod; citată pentru validarea schemelor utilizate împreună cu tipurile generate de Prisma., 2026, URL: https://zod.dev/?id=objects (accesat în 18.5.2026).

[4] Cockroach Labs, *Documentația CockroachDB*, Documentația CockroachDB, Documentație oficială CockroachDB; citată în planul de dezvoltare pentru analiza scalabilității orizontale., 2026, URL: https://www.cockroachlabs.com/docs/stable/postgresql-compatibility (accesat în 18.5.2026).

[5] Omar Dulaimi și colaboratorii, *Generatorul Prisma Zod*, Depozitul GitHub, Citată pentru generarea automată a schemelor de validare Zod pe baza modelelor Prisma., 2026, URL: https://github.com/omar-dulaimi/prisma-zod-generator (accesat în 18.5.2026).

[6] Devographics, *State of JavaScript 2024*, Sondaj anual al ecosistemului JavaScript, Sondaj anual al comunității JavaScript (peste 22.000 de respondenți); citat pentru tiparele de utilizare a SSR (59%) și SSG (46%), adopția TypeScript și complexitatea framework-urilor frontend., 2024, URL: https://2024.stateofjs.com/en-US/libraries/front-end-frameworks/ (accesat în 18.5.2026).

[7] Dezvoltatorii AnalogJS, *Analog – Meta-framework full-stack pentru Angular*, Documentația Analog, Documentație oficială Analog; citată pentru SSR, rutare bazată pe fișiere și integrarea Nitro/tRPC în aplicațiile Angular., 2026, URL: https://analogjs.org/docs/features/server-side-rendering (accesat în 18.5.2026).

[8] Dezvoltatorii AnalogJS, *Routing — Rutare bazată pe structura de fișiere*, Documentația Analog, Documentație oficială Analog; citată pentru rutarea *filesystem-based* (`*.page.ts`, rute dinamice cu parametri, colocarea logicii de server în `*.server.ts`)., 2026, URL: https://analogjs.org/docs/features/routing/overview (accesat în 18.5.2026).

[9] Echipa Google Angular, *Angular Material — Componente cu accesibilitate nativă*, Documentația Angular Material/CDK, Documentație oficială Angular Material; citată pentru setul de componente UI și pentru `Angular CDK`, consistent în privința accesibilității (*a11y*)., 2026, URL: https://material.angular.dev/ (accesat în 18.5.2026).

[10] Echipa Google Angular, *Prezentare generală a semnalelor (Signals Overview)*, Documentația Angular, Documentație oficială Angular; citată pentru modelul de reactivitate (Signals) utilizat în componente independente și în contextul randării pe server (SSR)., 2026, URL: https://angular.dev/guide/signals (accesat în 18.5.2026).

[11] Echipa Microsoft Playwright, *Testarea cu Playwright — Ghid introductiv*, Documentația Playwright, Documentație oficială Playwright; citată pentru testarea end-to-end adoptată în perioada iunie–iulie 2025., 2026, URL: https://playwright.dev/docs/test-global-setup-teardown (accesat în 18.5.2026).

[12] Echipa Nrwl, *Ce este Nx?*, Documentația Nx, Documentație oficială Nx; citată pentru orchestrarea monorepo-ului, rulările afectate, caching și graful de proiect., 2026, URL: https://nx.dev/docs/features/ci-features/affected (accesat în 18.5.2026).

[13] Echipa Nrwl, *Enforce Module Boundaries — Regulă ESLint pentru constrângeri arhitecturale*, Documentația Nx, Documentație oficială Nx; citată pentru regula `@nx/enforce-module-boundaries` care respinge la CI importurile care încalcă topologia dintre grupuri de biblioteci., 2026, URL: https://nx.dev/docs/features/enforce-module-boundaries (accesat în 18.5.2026).

[14] Echipa UnJS, *Nitro: Motorul de server pentru web*, Documentația Nitro, Documentație oficială Nitro; citată pentru execuția pe server, rutele API și mediile de implementare compatibile cu Analog SSR., 2026, URL: https://nitro.build/guide (accesat în 18.5.2026).

[15] Echipa Vite, *Ghidul Vite*, Documentația Vite, Documentație oficială Vite; citată pentru configurarea build-ului comună aplicațiilor și bibliotecilor și integrarea cu țintele Nx., 2026, URL: https://vitejs.dev/guide/why.html (accesat în 18.5.2026).

[16] Echipa Vitest, *Ghidul Vitest*, Documentația Vitest, Documentație oficială Vitest; citată pentru configurarea și utilizarea testelor unitare, folosită în proiect începând cu august 2025., 2026, URL: https://vitest.dev/guide/workspace.html (accesat în 18.5.2026).

[17] Eurostat, *Food waste: 132 kg per inhabitant in the EU in 2022*, Comunicat Eurostat, Comunicat oficial Eurostat; citat pentru date privind risipa alimentară per locuitor în Uniunea Europeană (132 kg/locuitor în 2022, 59,2 milioane tone total) și pentru ponderea gospodăriilor (54%) în risipa totală., 2024, URL: https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20240927-2 (accesat în 17.5.2026).

[18] Zheng Gao, Christian Bird și Earl T. Barr, „To Type or Not to Type: Quantifying Detectable Bugs in JavaScript", în *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, Studiu empiric care cuantifică, pe un eșantion de 400 de bug-uri publice din proiecte GitHub, proporția detectabilă prin tipizare statică (Flow și TypeScript); citat pentru fundamentarea empirică a avantajelor tipizării și pentru taxonomia bug-urilor nedetectabile., IEEE, 2017, pp. 758–769, DOI: 10.1109/ICSE.2017.75.

[19] Grupul Global de Dezvoltare PostgreSQL, *Documentația oficială PostgreSQL*, Documentația PostgreSQL, Documentație oficială PostgreSQL; citată pentru protocolul de legătură (*wire protocol*) pe care CockroachDB îl emulează pentru compatibilitate cu ecosistemul de unelte SQL., 2026, URL: https://www.postgresql.org/docs/current/protocol.html (accesat în 18.5.2026).

[20] JetBrains, *JavaScript and TypeScript Trends 2024*, Insights From the Developer Ecosystem Survey, Sondaj anual JetBrains; citat pentru creșterea adopției TypeScript (12% în 2017, 35% în 2024) și ponderea dezvoltatorilor predominant TypeScript (67%)., 2024, URL: https://blog.jetbrains.com/webstorm/2024/02/js-and-ts-trends-2024/ (accesat în 17.5.2026).

[21] Valeri Karpov, *The MEAN Stack: MongoDB, ExpressJS, AngularJS and Node.js*, MongoDB Blog, Articol original prin care Valeri Karpov a introdus termenul „MEAN stack"; citat ca referință istorică pentru arhitectura tradițională din ecosistemul JavaScript și pentru paralela conceptuală cu arhitectura PAN propusă în prezenta lucrare., Apr. 2013, URL: http://blog.mongodb.org/post/49262866911/the-mean-stack-mongodb-expressjs-angularjs-and (accesat în 17.5.2026).

[22] Parlamentul European, *Food waste in Europe: facts, EU policies and 2030 targets*, European Parliament Topics, Sinteză oficială a Parlamentului European; citată pentru cauzele comportamentale ale risipei alimentare la nivel de consumator (cumpărături impulsive, planificare insuficientă a meselor, gestionare deficitară a inventarului) și pentru obiectivele 2030., 2025, URL: https://www.europarl.europa.eu/topics/en/article/20240318STO19401/food-waste-in-europe-facts-eu-policies-and-2030-targets (accesat în 17.5.2026).

[23] David L. Parnas, „On the Criteria To Be Used in Decomposing Systems into Modules", în *Communications of the ACM* 15.12 (1972), Articol fondator al principiului ascunderii informației (information hiding) și al descompunerii modulare; citat pentru relația „uses" aciclică, pentru critica descompunerii pe diagrama de flux și pentru ascunderea deciziilor de proiectare schimbabile., pp. 1053–1058, DOI: 10.1145/361598.361623.

[24] Prisma Labs, *Conectare Prisma la CockroachDB*, Documentația Prisma ORM, Documentație oficială Prisma; citată pentru integrarea cu `CockroachDB` prin protocolul de legătură PostgreSQL și pentru particularitățile suportului distribuit., 2026, URL: https://www.prisma.io/docs/orm/overview/databases/cockroachdb (accesat în 18.5.2026).

[25] Prisma Labs, *Prisma Client — Generarea clientului tipizat și interogări*, Documentația Prisma ORM, Documentație oficială Prisma; citată pentru generarea clientului TypeScript din `schema.prisma`, gestionarea migrațiilor versionate și inferența tipurilor pentru fiecare model., 2026, URL: https://www.prisma.io/docs/orm/prisma-client (accesat în 18.5.2026).

[26] Prisma Labs, *Tranzacții și operații în lot*, Documentația Prisma Client, Documentație oficială Prisma; citată pentru `prisma.$transaction`, gestionarea conflictelor *serializable* (cod `P2034`) și pentru strategiile de reluare automată în baze distribuite., 2026, URL: https://www.prisma.io/docs/orm/prisma-client/queries/transactions (accesat în 18.5.2026).

[27] Programul Națiunilor Unite pentru Mediu, *Food Waste Index Report 2024*, Raport oficial UNEP, Raport UNEP, co-autor WRAP; citat pentru date globale privind risipa alimentară la nivel de gospodărie (60% din risipa totală, 631 milioane tone în 2022) și pentru obiectivul SDG 12.3 de înjumătățire a risipei alimentare până în 2030., 2024, URL: https://www.unep.org/resources/publication/food-waste-index-report-2024 (accesat în 17.5.2026).

[28] Proiectul tRPC, *Documentația tRPC — API-uri type-safe end-to-end*, Documentația tRPC, Documentație oficială tRPC; citată pentru contractele de tipuri între client și server și pentru structura router/context., 2026, URL: https://trpc.io/docs/server/procedures (accesat în 18.5.2026).

[29] Proiectul tRPC, *Middlewares — Constructori de proceduri și compunere de autorizare*, Documentația tRPC, Documentație oficială tRPC; citată pentru constructorii compoziționali de proceduri (`publicProcedure`, `protectedProcedure`) și pentru verificarea identității în context., 2026, URL: https://trpc.io/docs/server/middlewares (accesat în 18.5.2026).

[30] Redis, *Documentația Redis*, Documentația Redis, Documentație oficială Redis; citată în planul de dezvoltare pentru strategiile de caching și gestionare a sesiunilor de interogare., 2026, URL: https://redis.io/docs/latest/ (accesat în 18.5.2026).

[31] Karin Schanes, Karin Dobernig și Burcu Gözet, „Food waste matters — A systematic review of household food waste practices and their policy implications”, în *Journal of Cleaner Production* 182 (2018), Sinteză sistematică (60 de studii empirice) asupra practicilor casnice de risipă alimentară; citată pentru cauzele comportamentale la nivel de consumator și pentru rolul planificării, al verificării inventarului și al aplicațiilor digitale în prevenție., pp. 978–991, DOI: 10.1016/j.jclepro.2018.02.030.

[32] Jacopo Soldani, Damian Andrew Tamburri și Willem-Jan Van Den Heuvel, „The Pains and Gains of Microservices: A Systematic Grey Literature Review”, în *Journal of Systems and Software* 146 (2018), Analiză sistematică a literaturii industriale (51 de studii) asupra durerilor și câștigurilor arhitecturii pe microservicii; citată pentru poziționarea arhitecturii PAN ca monolit modular în spațiul de compromisuri., pp. 215–232, DOI: 10.1016/j.jss.2018.09.082.

[33] Vercel, *Vercel CLI — Command Line Interface*, Documentația Vercel, Documentație oficială pentru gestionarea proiectelor și a variabilelor de mediu în Vercel., 2026, URL: https://vercel.com/docs/cli/env (accesat în 18.5.2026).